Bases de datos relacionales · Práctica de bases de datos

SQL: resumen

1. **Sublenguajes de SQL**: Los sublenguajes de SQL · DDL, DML, DCL y TCL comparados
2. **Oracle y FreeSQL**: Practicar con Oracle en FreeSQL · Oracle frente a otros dialectos de SQL
3. **Tablas y tipos**: DDL: definir el esquema · CREATE TABLE paso a paso · Tipos de datos de Oracle · Columnas identidad
4. **Modificar y eliminar**: ALTER, DROP y TRUNCATE · El diccionario de datos, COMMENT y RENAME
5. **Restricciones**: PRIMARY KEY, UNIQUE y NOT NULL · FOREIGN KEY y acciones referenciales · CHECK y DEFAULT · Gestionar restricciones con ALTER TABLE
6. **Objetos del esquema**: Vistas · Secuencias y sinónimos
7. **Consultas**: SELECT: columnas, expresiones y alias · Ordenar resultados con ORDER BY · Filtrar con WHERE, AND, OR y NOT · BETWEEN, IN y LIKE · Trabajar con NULL · Combinar tablas con INNER JOIN
8. **Combinar tablas**: Combinaciones externas: LEFT, RIGHT y FULL · CROSS JOIN, autocombinaciones, USING y NATURAL
9. **Agregación**: Funciones de agregado: COUNT, SUM, AVG, MIN y MAX · GROUP BY y HAVING
10. **Subconsultas y conjuntos**: Subconsultas en WHERE, FROM y SELECT · Subconsultas de varias filas: IN, ANY y ALL · Subconsultas correlacionadas y EXISTS · Operadores de conjuntos: UNION, INTERSECT y MINUS · WITH: expresiones de tabla comunes
11. **Funciones**: Funciones de texto · Funciones numéricas y de fecha · Conversión, funciones para NULL, CASE y DECODE
12. **Modificar datos**: DML: leer y modificar filas · INSERT en la práctica · UPDATE y DELETE en la práctica
13. **DCL: usuarios y privilegios**: DCL: usuarios, roles y privilegios · Privilegios de sistema, privilegios de objeto y roles
14. **TCL: transacciones**: TCL: transacciones · Commit implícito y autocommit
15. **PL/SQL**: Bloques PL/SQL, variables y SELECT INTO · Control de flujo: IF, CASE y bucles · Cursores: recorrer las filas una a una · Excepciones: gestionar los errores · Procedimientos y funciones almacenados · Disparadores (triggers) · Paquetes
16. **Índices**: Qué es un índice · Índices compuestos y de cobertura · Lo que cuestan los índices · Tipos de índice
17. **Agrupamiento**: Índices agrupados y no agrupados · Clústeres de tablas en Oracle (y CLUSTER en PostgreSQL)
18. **Particionado**: Particionado horizontal · Poda de particiones y mantenimiento · Particionado vertical y sharding
19. **Eficiencia**: Leer un plan de ejecución · Condiciones que pueden usar un índice · Escribir consultas eficientes

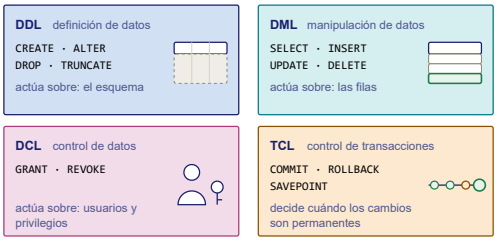
Sublenguajes de SQL

Los sublenguajes de SQL

SQL es un solo lenguaje con varios sublenguajes, agrupados según lo que hace cada sentencia: definir la estructura, trabajar con los datos, controlar quién puede hacer qué y agrupar los cambios en transacciones.

Los cuatro grupos

Sublenguaje	Sentencias	Sobre qué actúa
DDL · definición de datos	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`, `COMMENT`	El esquema: tablas, columnas, restricciones, índices, vistas
DML · manipulación de datos	`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `MERGE`	Las filas almacenadas en las tablas
DCL · control de datos	`GRANT`, `REVOKE`	Usuarios, roles y sus privilegios
TCL · control de transacciones	`COMMIT`, `ROLLBACK`, `SAVEPOINT`, `SET TRANSACTION`	Cuándo los cambios pasan a ser permanentes o se deshacen



Cada sublenguaje actúa sobre algo distinto.

> **Ejemplo.** `CREATE TABLE student (…)` es DDL; `INSERT INTO student VALUES (…)` es DML; `GRANT SELECT ON student TO teacher` es DCL; `COMMIT` es TCL.

> **Error frecuente.** Pensar que `TRUNCATE` es lo mismo que `DELETE` sin WHERE. `TRUNCATE` es DDL: vacía la tabla de golpe, normalmente no admite filtro y en muchos SGBD confirma la transacción de forma implícita.

DDL, DML, DCL y TCL comparados

Los sublenguajes se distinguen por lo que cambian y por si se puede deshacer. DML cambia filas dentro de una transacción que tú confirmas o deshaces; DDL y DCL cambian el diccionario de datos y confirman en el

acto; TCL termina la transacción.

Los cuatro sublenguajes, uno al lado del otro

	DDL	DML	DCL	TCL
Actúa sobre	El esquema (el diccionario de datos)	Las filas de las tablas	Usuarios, roles y privilegios	La transacción abierta
Sentencias	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`, `COMMENT`	`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `MERGE`	`GRANT`, `REVOKE`	`COMMIT`, `ROLLBACK`, `SAVEPOINT`
¿Dentro de la transacción?	No: confirma antes y después	Sí: espera al COMMIT	No: confirma antes y después	La termina, o marca un punto en ella
¿ROLLBACK la deshace?	No	Sí, hasta el COMMIT	No	·

Tres maneras de vaciar una tabla

	`DELETE FROM t`	`TRUNCATE TABLE t`	`DROP TABLE t`
Sublenguaje	DML	DDL	DDL
Elimina	Las filas que cumplen el WHERE (todas si no hay WHERE)	Todas las filas	Todas las filas y la propia tabla
Después	La tabla sigue ahí	La tabla sigue ahí, vacía	La tabla ya no existe (va a la papelera, salvo con PURGE)
ROLLBACK	Lo deshace	No lo puede deshacer	No lo puede deshacer (`FLASHBACK TABLE` puede recuperarla)
En una tabla grande	Lento: registra cada fila	Rápido	Rápido
En una tabla padre	Falla solo con las filas que aún tienen hijas	Falla (ORA-02266) mientras una clave ajena activa apunte a ella	Falla (ORA-02449) salvo con `CASCADE CONSTRAINTS`

Ejemplo que puedes ejecutar en la web

```
-- DML: forma parte de la transacción, así que ROLLBACK la deshace
DELETE FROM enrolment;
SELECT COUNT(*) AS after_delete FROM enrolment;
ROLLBACK;
SELECT COUNT(*) AS after_delete_and_rollback FROM enrolment;
-- DDL: TRUNCATE confirma, así que no hay vuelta atrás
TRUNCATE TABLE enrolment;
ROLLBACK;
SELECT COUNT(*) AS after_truncate_and_rollback FROM enrolment;
-- DDL otra vez: DROP elimina la propia tabla
DROP TABLE enrolment;
SELECT table_name FROM user_tables ORDER BY table_name;
```

> **Ejemplo.** En una práctica vacías una copia de una tabla: `DELETE` mientras aún quieras poder deshacerlo, `TRUNCATE` cuando estés seguro y `DROP` cuando ya no necesites la copia.

> **Error frecuente.** Creer que ROLLBACK deshace «lo último que hice». Deshace solo el DML de la transacción abierta; un CREATE, ALTER, TRUNCATE o GRANT ejecutado en medio ya ha confirmado todo lo anterior.

Oracle y FreeSQL

Practicar con Oracle en FreeSQL

FreeSQL (freesql.com) es la hoja de trabajo en línea y gratuita de Oracle: escribes SQL de Oracle en el navegador y se ejecuta en una base de datos Oracle real, sin instalar nada. Es donde se practica en este curso.

- Ejecutar una sentencia: coloca el cursor en ella y ejecuta. Ejecutar todo el script: usa la opción de script de la hoja de trabajo.
- Tu esquema se conserva entre sesiones: borra lo que ya no necesites.
- La simulación de esta página no admite PL/SQL, particionado, secuencias ni privilegios: para eso, FreeSQL es el único sitio donde ejecutarlo.

> **Error frecuente.** Fiarse solo de la simulación. Traduce un subconjunto de Oracle a otro motor distinto: cuando su resultado te sorprenda, compruébalo en FreeSQL.

Oracle frente a otros dialectos de SQL

SQL es un estándar, pero cada SGBD tiene su propio dialecto. La mayoría de los errores de quien aprendió SQL en otro sitio vienen de unas pocas diferencias: limitar filas, tipos, fechas, cadenas vacías y transacciones.

Oracle frente a SQLite, PostgreSQL y MySQL

Tema	Oracle	Otros
Primeras n filas	`FETCH FIRST n ROWS ONLY`	`LIMIT n`
Diferencia de conjuntos	`MINUS`	`EXCEPT`
Tipo de texto	`VARCHAR2(n)`, `CLOB`	`TEXT`, `VARCHAR(n)`
Números	`NUMBER(p, s)`	`INTEGER`, `NUMERIC`, `REAL`
Numeración automática	`GENERATED ALWAYS AS IDENTITY`	`AUTOINCREMENT`, `SERIAL`, `AUTO_INCREMENT`
Fecha actual	`SYSDATE`	`date('now')`, `CURRENT_DATE`, `NOW()`
Literal de fecha	`DATE '2024-01-31'`, `TO_DATE(…)`	Una cadena de texto
Sustituir NULL	`NVL(a, b)`, `COALESCE`	`IFNULL(a, b)`
Cadena vacía	Es **NULL**	Es una cadena vacía
SELECT sin tabla	`SELECT 1 FROM dual`	`SELECT 1`
Transacciones	Empiezan con el primer DML; el DDL confirma	`BEGIN` / `START TRANSACTION`
Diccionario de datos	`USER_TABLES`, `USER_TAB_COLUMNS`	`sqlite_master`, `information_schema`
Plan de ejecución	`EXPLAIN PLAN FOR` + `DBMS_XPLAN`	`EXPLAIN`

> **Error frecuente.** Escribir una comparación como `WHERE signup_date >= '2024-01-01'` contra una columna DATE. Depende del formato de fecha de la sesión y a menudo falla (ORA-01861); escribe `DATE '2024-01-01'`.

Tablas y tipos

DDL: definir el esquema

`CREATE` crea un objeto, `ALTER` lo modifica y `DROP` lo elimina, datos incluidos. Con DDL el modelo lógico se convierte en tablas reales.

- Tipos de Oracle: `VARCHAR2(n)` para texto, `NUMBER(p, s)` para todos los números, `DATE` (también guarda la hora), `TIMESTAMP`, `CLOB`, `BLOB`. Desde 23ai existe también `BOOLEAN`.
- Las claves autonuméricas usan `GENERATED ALWAYS AS IDENTITY`; no existe AUTOINCREMENT.
- Cada sentencia DDL actualiza el diccionario de datos, que se puede consultar: `USER_TABLES`, `USER_TAB_COLUMNS`, `USER_INDEXES`, `USER_CONSTRAINTS`.

Ejemplo que puedes ejecutar en la web

```
CREATE TABLE course (
  course_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title     VARCHAR2(40) NOT NULL,
  credits   NUMBER(3,1) NOT NULL
);
ALTER TABLE course ADD (semester NUMBER(1));
INSERT INTO course (title, credits, semester) VALUES ('Databases', 6, 1);
SELECT * FROM course;
SELECT table_name FROM user_tables;
DROP TABLE course;
SELECT table_name FROM user_tables;
```

Mira el vídeo en la web, en la sección SQL.

> **Error frecuente.** Borrar y volver a crear una tabla para cambiar una columna. `ALTER TABLE` conserva las filas; `DROP` las pierde.

CREATE TABLE paso a paso

Un CREATE TABLE da a cada columna un nombre, un tipo y sus restricciones. Una restricción se puede escribir junto a su columna (en línea) o tras la última columna (fuera de línea), donde puede abarcar varias columnas.

- Pon nombre a tus restricciones (`CONSTRAINT fk_player_team …`): los mensajes de error y el diccionario de datos muestran entonces un nombre legible en lugar de algo como `SYS_C008123`.
- Una columna de clave ajena tiene el mismo tipo que la clave a la que apunta.
- En Oracle `DEFAULT` va antes de `NOT NULL`: `pos CHAR(1) DEFAULT 'M' NOT NULL`.

Ejemplo que puedes ejecutar en la web

```
CREATE TABLE team (
  team_id NUMBER(4)    PRIMARY KEY,
  name    VARCHAR2(40) NOT NULL UNIQUE,
  city    VARCHAR2(30)
);
CREATE TABLE player (
  player_id NUMBER(6)    GENERATED ALWAYS AS IDENTITY,
  full_name VARCHAR2(60) NOT NULL,
  shirt     NUMBER(2)    CHECK (shirt BETWEEN 1 AND 99),
  pos       CHAR(1)      DEFAULT 'M' NOT NULL,
  born      DATE,
  team_id   NUMBER(4)    NOT NULL,
  CONSTRAINT pk_player PRIMARY KEY (player_id),
  CONSTRAINT fk_player_team FOREIGN KEY (team_id) REFERENCES team (team_id),
  CONSTRAINT uq_team_shirt UNIQUE (team_id, shirt),
  CONSTRAINT ck_player_pos CHECK (pos IN ('G', 'D', 'M', 'F'))
);
INSERT INTO team VALUES (1, 'Lions', 'Madrid');
INSERT INTO player (full_name, shirt, pos, born, team_id) VALUES ('Lucia Mora', 9, 'F', DATE '2004-03-12', 1);
INSERT INTO player (full_name, shirt, team_id) VALUES ('Pablo Ruiz', 4, 1);
SELECT * FROM player;
SELECT column_name FROM user_tab_columns WHERE table_name = 'PLAYER' ORDER BY column_id;
INSERT INTO player (full_name, shirt, team_id) VALUES ('Nobody', 7, 99);   -- el equipo 99 no existe
```

> **Ejemplo.** `team` es la tabla padre y `player` la hija: cada jugador pertenece a un equipo que existe, y un dorsal no se puede repetir dentro del mismo equipo.

> **Error frecuente.** Crear la tabla hija antes que su padre. Su FOREIGN KEY falla con ORA-00942 (table or view does not exist), porque la tabla referenciada aún no está.

Tipos de datos de Oracle

Cada columna tiene un tipo que fija lo que puede guardar. Oracle traduce los tipos del estándar SQL a unos pocos propios: VARCHAR2 para texto, NUMBER para cualquier número, DATE y TIMESTAMP para instantes de tiempo, CLOB y BLOB para objetos grandes.

- `VARCHAR2(n)` cuenta **bytes** por defecto. Con letras como la «ñ» o las tildes, escribe `VARCHAR2(n CHAR)` para contar caracteres.
- En Oracle la cadena vacía `''` es NULL.
- `USER_TAB_COLUMNS` (o `DESCRIBE tabla` en SQL Developer) muestra el tipo de cada columna.

Tipos del estándar SQL y su equivalente en Oracle

Clase de dato	Estándar SQL	Oracle
Texto de longitud fija	`CHAR(n)`	`CHAR(n)`
Texto de longitud variable	`VARCHAR(n)`, `CHARACTER VARYING(n)`	`VARCHAR2(n)`
Texto nacional (Unicode)	`NCHAR(n)`, `NCHAR VARYING(n)`	`NCHAR(n)`, `NVARCHAR2(n)`
Enteros	`SMALLINT`, `INTEGER`, `BIGINT`	`NUMBER(n)`, o `NUMBER`
Decimales exactos	`NUMERIC(p,s)`, `DECIMAL(p,s)`	`NUMBER(p,s)`
Números aproximados	`FLOAT`, `REAL`, `DOUBLE PRECISION`	`NUMBER`, `BINARY_FLOAT`, `BINARY_DOUBLE`
Fecha	`DATE`	`DATE` (con la hora del día)

Clase de dato	Estándar SQL	Oracle
Fecha y hora	`TIMESTAMP`	`TIMESTAMP`
Intervalo	`INTERVAL YEAR TO MONTH`, `INTERVAL DAY TO SECOND`	Los mismos
Booleano	`BOOLEAN`	Ninguno en 21c: `CHAR(1)` o `NUMBER(1)`
Binario	`BIT`, `BIT VARYING`	`RAW(n)`
Texto largo	`CLOB`	`CLOB` (`LONG` está obsoleto)
Binario largo	`BLOB`	`BLOB`, `BFILE` (`LONG RAW` está obsoleto)
Dirección de una fila	·	`ROWID`

Oracle · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS product PURGE;
CREATE TABLE product (
  product_id NUMBER(6)         PRIMARY KEY,
  name       VARCHAR2(40 CHAR) NOT NULL,
  country    CHAR(2),
  price      NUMBER(6,2)       NOT NULL,
  in_stock   CHAR(1)           DEFAULT 'Y' NOT NULL CHECK (in_stock IN ('Y', 'N')),
  added      DATE              DEFAULT SYSDATE NOT NULL,
  notes      CLOB
);
INSERT INTO product (product_id, name, country, price) VALUES (1, 'Café molido', 'CO', 4.256);
SELECT product_id, name, price, TO_CHAR(added, 'YYYY-MM-DD HH24:MI:SS') AS added FROM product;   -- 4.26
SELECT column_name, data_type, data_length, data_precision, data_scale, nullable
FROM user_tab_columns WHERE table_name = 'PRODUCT' ORDER BY column_id;
INSERT INTO product (product_id, name, price) VALUES (2, 'Un nombre demasiado largo para cuarenta letras', 1);   -
- ORA-12899
INSERT INTO product (product_id, name, price) VALUES (3, 'Té', 12345.5);   -- ORA-01438: más de 4 dígitos antes
del punto
```

Ejemplo. `price NUMBER(6,2)` guarda hasta 9999.99, `country CHAR(2)` guarda códigos como 'ES' y `added DATE DEFAULT SYSDATE` registra el día y la hora en que se creó la fila.

Error frecuente. Guardar fechas o números como texto. '10/02/2026' no se puede comparar ni ordenar como fecha, y como texto '9' es mayor que '10'.

Columnas identidad

Una columna identidad numera las filas sola: en cada INSERT Oracle toma el siguiente valor de una secuencia oculta. ALWAYS, BY DEFAULT y BY DEFAULT ON NULL deciden si además puedes dar tú el valor.

- Usa ALWAYS para las claves subrogadas: nadie puede escribir un número que choque después con uno generado.
- BY DEFAULT sirve para cargar datos antiguos con sus números de siempre; pon START WITH por encima del mayor, o los valores generados chocarán con ellos.
- Oracle crea la secuencia oculta (`ISEQ$$_…`) y la borra con la tabla. `USER_TAB_IDENTITY_COLS` lista las columnas identidad.
- En la simulación del navegador una columna identidad se convierte en un INTEGER autonumerado; los modos y START WITH solo funcionan en FreeSQL.

Qué hace cada modo con el INSERT

	El INSERT la omite	El INSERT da NULL	El INSERT da un valor
`GENERATED ALWAYS`	Oracle la numera	Error (ORA-32795)	Error (ORA-32795)
`GENERATED BY DEFAULT`	Oracle la numera	Error (ORA-01400)	Se acepta
`GENERATED BY DEFAULT ON NULL`	Oracle la numera	Oracle la numera	Se acepta

Oracle · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS identity_demo PURGE;
CREATE TABLE identity_demo (
  id          NUMBER GENERATED BY DEFAULT ON NULL AS IDENTITY (START WITH 100 INCREMENT BY 10) PRIMARY KEY,
  description VARCHAR2(100) NOT NULL
);
INSERT INTO identity_demo (description) VALUES ('Omitida: Oracle pone 100');
```

```
INSERT INTO identity_demo (id, description) VALUES (NULL, 'NULL: ON NULL pone 110');
INSERT INTO identity_demo (id, description) VALUES (5, 'Un valor propio: se acepta');
SELECT * FROM identity_demo ORDER BY id;

DROP TABLE IF EXISTS ticket PURGE;
CREATE TABLE ticket (
  ticket_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer  VARCHAR2(30) NOT NULL
);
INSERT INTO ticket (customer) VALUES ('Ana');                  -- ticket 1
SELECT table_name, column_name, generation_type, identity_options FROM user_tab_identity_cols;
INSERT INTO ticket (ticket_id, customer) VALUES (7, 'Luis');   -- ORA-32795: no se puede insertar en una columna
identidad GENERATED ALWAYS
```

Ejemplo. Con `GENERATED BY DEFAULT ON NULL AS IDENTITY (START WITH 100 INCREMENT BY 10)` la primera fila recibe 100, una fila insertada con `id = NULL` recibe 110 y una insertada con `id = 5` conserva su 5.

Error frecuente. Dar un valor a una columna GENERATED ALWAYS, por ejemplo al reutilizar un script antiguo con `VALUES (1, …)`. Deja la columna fuera de la lista de columnas.

Modificar y eliminar

ALTER, DROP y TRUNCATE

ALTER TABLE cambia una tabla que ya tiene datos: añade, modifica, renombra o elimina columnas y restricciones. DROP TABLE elimina la tabla; TRUNCATE la vacía y conserva su estructura.

- `ADD`, `RENAME COLUMN` y `DROP COLUMN` funcionan en la simulación del navegador; `MODIFY`, `ADD CONSTRAINT` y la papelera, solo en FreeSQL (el script estático de abajo).
- `ALTER TABLE … ADD CONSTRAINT` comprueba antes las filas que ya existen: falla si alguna incumple la regla.
- Cada ALTER, DROP y TRUNCATE confirma la transacción abierta.

Oracle · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS member PURGE;
CREATE TABLE member (
  member_id NUMBER(4) PRIMARY KEY,
  full_name VARCHAR2(40) NOT NULL,
  email     VARCHAR2(60)
);
INSERT INTO member VALUES (1, 'Ana Ruiz', 'ana@uni.es');
INSERT INTO member VALUES (2, 'Luis Gil', 'luis@uni.es');
COMMIT;

ALTER TABLE member MODIFY (full_name VARCHAR2(80));         -- alarga la columna
ALTER TABLE member MODIFY (email NOT NULL);                 -- funciona: todas las filas tienen email
ALTER TABLE member ADD CONSTRAINT uq_member_email UNIQUE (email);
ALTER TABLE member DROP CONSTRAINT uq_member_email;

DROP TABLE member;                                          -- a la papelera de reciclaje
SELECT object_name, original_name FROM user_recyclebin;
FLASHBACK TABLE member TO BEFORE DROP;                      -- vuelve, con sus filas
SELECT * FROM member;
DROP TABLE member PURGE;                                    -- desaparece para siempre
```

Ejemplo que puedes ejecutar en la web

```
CREATE TABLE member (
  member_id NUMBER(4) PRIMARY KEY,
  full_name VARCHAR2(40) NOT NULL
);
INSERT INTO member VALUES (1, 'Ana Ruiz');
INSERT INTO member VALUES (2, 'Luis Gil');
ALTER TABLE member ADD (email VARCHAR2(60), fee NUMBER(5,2) DEFAULT 20 NOT NULL);
ALTER TABLE member RENAME COLUMN full_name TO member_name;
SELECT * FROM member;
```

```
ALTER TABLE member DROP COLUMN fee;
-- DELETE se puede deshacer...
DELETE FROM member;
ROLLBACK;
SELECT COUNT(*) AS after_delete_and_rollback FROM member;
-- ...TRUNCATE no: es DDL y confirma
TRUNCATE TABLE member;
ROLLBACK;
SELECT COUNT(*) AS after_truncate_and_rollback FROM member;
DROP TABLE member PURGE;
```

Mira el vídeo en la web, en la sección SQL.

> **Ejemplo.** `ALTER TABLE member ADD (email VARCHAR2(60))` añade una columna a una tabla que ya tiene socios: su email es NULL hasta que alguien lo rellene.

> **Error frecuente.** Usar TRUNCATE «para probar algo» con datos reales. No se puede deshacer; ante la duda, usa DELETE y comprueba antes del COMMIT.

El diccionario de datos, COMMENT y RENAME

Cada sentencia DDL queda registrada en el diccionario de datos, un conjunto de vistas de solo lectura que describen tu esquema: USER_TABLES, USER_TAB_COLUMNS, USER_CONSTRAINTS y otras. COMMENT ON documenta allí tablas y columnas, y RENAME cambia un nombre.

- El diccionario es de solo lectura: cambia con DDL, nunca con INSERT o UPDATE.
- La simulación del navegador conoce `USER_TABLES` y `USER_TAB_COLUMNS`; las demás vistas, COMMENT y RENAME TO solo funcionan en FreeSQL.

Vistas del diccionario que vas a usar

Vista	Qué lista
`USER_TABLES`	Tus tablas
`USER_TAB_COLUMNS`	Sus columnas: tipo, longitud, precisión y si admiten NULL
`USER_CONSTRAINTS`	Las restricciones: nombre, tipo (P, R, U, C), estado y, en una clave ajena, la clave a la que apunta
`USER_CONS_COLUMNS`	Las columnas de cada restricción
`USER_INDEXES`, `USER_IND_COLUMNS`	Los índices y sus columnas
`USER_VIEWS`, `USER_SEQUENCES`, `USER_SYNONYMS`	Los demás objetos del esquema
`USER_OBJECTS`	Todos los objetos, con su tipo y su estado (VALID o INVALID)
`USER_RECYCLEBIN`	Las tablas borradas que FLASHBACK aún puede recuperar

Oracle · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS player PURGE;
DROP TABLE IF EXISTS team PURGE;
CREATE TABLE team (
  team_id NUMBER(4)    CONSTRAINT pk_team PRIMARY KEY,
  name    VARCHAR2(40) NOT NULL
);
CREATE TABLE player (
  player_id NUMBER(6)    CONSTRAINT pk_player PRIMARY KEY,
  full_name VARCHAR2(60) NOT NULL,
  team_id   NUMBER(4)    CONSTRAINT fk_player_team REFERENCES team (team_id)
);
SELECT column_name, data_type, data_length, nullable
FROM user_tab_columns WHERE table_name = 'PLAYER' ORDER BY column_id;
SELECT constraint_name, constraint_type, r_constraint_name, status
FROM user_constraints WHERE table_name = 'PLAYER';
SELECT constraint_name, column_name FROM user_cons_columns WHERE table_name = 'PLAYER';

COMMENT ON TABLE player IS 'Una fila por jugador inscrito';
COMMENT ON COLUMN player.team_id IS 'El equipo al que pertenece el jugador';
SELECT column_name, comments FROM user_col_comments WHERE table_name = 'PLAYER' AND comments IS NOT NULL;
```

```
ALTER TABLE player RENAME COLUMN full_name TO player_name;
RENAME player TO squad_member;              -- lo mismo que ALTER TABLE player RENAME TO squad_member
SELECT object_name, object_type, status FROM user_objects WHERE object_type = 'TABLE';
```

Ejemplo que puedes ejecutar en la web

```
SELECT table_name FROM user_tables ORDER BY table_name;
SELECT column_name FROM user_tab_columns WHERE table_name = 'COURSE' ORDER BY column_id;
ALTER TABLE course RENAME COLUMN title TO course_title;
SELECT column_name FROM user_tab_columns WHERE table_name = 'COURSE' ORDER BY column_id;
SELECT course_id, course_title FROM course;
```

> **Ejemplo.** `SELECT constraint_name, constraint_type FROM user_constraints WHERE table_name = 'ENROLMENT'` muestra la clave primaria (P), las dos claves ajenas (R) y el CHECK de la nota (C) del esquema de ejemplo.

> **Error frecuente.** Buscar una tabla por su nombre en minúsculas. Los nombres sin comillas se guardan en **mayúsculas**, así que `WHERE table_name = 'player'` no encuentra nada: escribe `'PLAYER'`.

Restricciones

PRIMARY KEY, UNIQUE y NOT NULL

La clave primaria identifica cada fila: única y nunca NULL. UNIQUE prohíbe duplicados en otras claves candidatas y NOT NULL hace obligatoria una columna.

- Clave compuesta: `PRIMARY KEY (student_id, course_id)`.
- La última sentencia del ejemplo falla a propósito: lee el mensaje de error.

Ejemplo que puedes ejecutar en la web

```
INSERT INTO student VALUES ('S04', 'Marta Paz', NULL);   -- se admite un segundo correo NULL
SELECT * FROM student;
INSERT INTO student VALUES ('S05', 'Ana Ruiz', 'ana@uni.es');  -- correo duplicado
```

Mira el vídeo en la web, en la sección SQL.

> **Error frecuente.** Usar solo un identificador artificial y olvidar la clave natural. Con `id` como PK y sin `UNIQUE (email)`, el mismo estudiante se puede insertar dos veces.

FOREIGN KEY y acciones referenciales

Una clave ajena solo acepta valores que existen en la clave referenciada. Su acción referencial indica qué ocurre con las filas hijas cuando se borra la fila padre o cambia su clave.

- CASCADE encaja con partes que no tienen sentido solas (una matrícula sin su estudiante, una línea de pedido sin su pedido).
- Sin acción (la opción por defecto) protege los datos de referencia (una asignatura, un departamento).
- Insertar una fila hija sin padre falla con ORA-02291 (clave padre no encontrada); borrar un padre con hijas falla con ORA-02292 (registro hijo encontrado).

Ejemplo que puedes ejecutar en la web

```
DELETE FROM student WHERE student_id = 'S01';     -- CASCADE: también se van las matrículas de Ana
SELECT * FROM enrolment ORDER BY student_id;
DELETE FROM course WHERE course_id = 'C10';       -- C10 aún tiene matrículas: rechazado
```

> **Error frecuente.** Usar CASCADE en todas partes. Borrar un departamento podría entonces borrar sin avisar sus asignaturas y todas sus matrículas.

CHECK y DEFAULT

CHECK establece una condición que debe cumplir cada fila, como una nota entre 0 y 10. DEFAULT da su valor a

una columna cuando el INSERT no indica ninguno.

- Un CHECK se supera cuando la condición es verdadera **o desconocida** (NULL); añade NOT NULL si el valor es obligatorio.

Ejemplo que puedes ejecutar en la web

```
CREATE TABLE loan (
  loan_id   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  book      VARCHAR2(60) NOT NULL,
  loan_date DATE DEFAULT SYSDATE NOT NULL,
  days      NUMBER(2) DEFAULT 15 NOT NULL,
  status    VARCHAR2(10) DEFAULT 'open' NOT NULL,
  CONSTRAINT days_range CHECK (days BETWEEN 1 AND 60),
  CONSTRAINT status_values CHECK (status IN ('open', 'returned', 'lost'))
);
INSERT INTO loan (book) VALUES ('Don Quixote');
INSERT INTO loan (book, days, status) VALUES ('Ficciones', 30, 'returned');
SELECT * FROM loan;
INSERT INTO loan (book, days) VALUES ('Hopscotch', 90);       -- incumple days_range
```

> **Error frecuente.** Comprobar los rangos solo en la aplicación. Los datos cargados con un script o desde otro programa se saltan entonces la regla.

Gestionar restricciones con ALTER TABLE

Las restricciones se pueden añadir, renombrar, eliminar, desactivar y activar cuando la tabla ya existe. Las restricciones con nombre son fáciles de gestionar, y Oracle comprueba las filas que ya hay antes de aceptar una restricción nueva o reactivada.

- Pon nombre a todas las restricciones: prefijos como `pk_`, `fk_`, `uq_` y `ck_` dicen el tipo de un vistazo, y el mensaje de error muestra el nombre.
- `USER_CONSTRAINTS.STATUS` dice si una restricción está ENABLED o DISABLED.
- La simulación del navegador no puede añadir, desactivar ni eliminar restricciones: ejecuta el script en FreeSQL.

Oracle · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS loan PURGE;
DROP TABLE IF EXISTS book PURGE;
CREATE TABLE book (book_id NUMBER(5), title VARCHAR2(60));
CREATE TABLE loan (loan_id NUMBER(6), book_id NUMBER(5), days NUMBER(2));
-- Las restricciones, añadidas después de las tablas
ALTER TABLE book ADD CONSTRAINT pk_book PRIMARY KEY (book_id);
ALTER TABLE book MODIFY (title NOT NULL);
ALTER TABLE loan ADD CONSTRAINT pk_loan PRIMARY KEY (loan_id);
ALTER TABLE loan ADD CONSTRAINT fk_loan_book FOREIGN KEY (book_id) REFERENCES book (book_id);
ALTER TABLE loan ADD CONSTRAINT ck_loan_days CHECK (days BETWEEN 1 AND 60);
INSERT INTO book VALUES (1, 'Ficciones');
INSERT INTO loan VALUES (1, 1, 15);

-- Apagar una regla, cargar y volver a encenderla
ALTER TABLE loan DISABLE CONSTRAINT ck_loan_days;
INSERT INTO loan VALUES (2, 1, 90);                   -- se acepta mientras la regla está apagada
ALTER TABLE loan ENABLE CONSTRAINT ck_loan_days;      -- ORA-02293: no se puede validar, se incumple la
restricción
UPDATE loan SET days = 60 WHERE loan_id = 2;
ALTER TABLE loan ENABLE CONSTRAINT ck_loan_days;      -- ahora funciona
SELECT constraint_name, constraint_type, status FROM user_constraints WHERE table_name = 'LOAN';

ALTER TABLE loan RENAME CONSTRAINT ck_loan_days TO ck_loan_days_range;
ALTER TABLE book DROP PRIMARY KEY;                     -- ORA-02273: hay claves ajenas que la referencian
ALTER TABLE book MODIFY (title VARCHAR2(5));           -- ORA-01441: algún valor es demasiado grande
ALTER TABLE book READ ONLY;
INSERT INTO book VALUES (2, 'Rayuela');                -- ORA-12081: operación de actualización no permitida en la
tabla
ALTER TABLE book READ WRITE;
```

> **Ejemplo.** Un script crea primero todas las tablas y luego ejecuta `ALTER TABLE loan ADD CONSTRAINT fk_loan_book FOREIGN KEY (book_id) REFERENCES book (book_id)` para cada clave ajena, así que las tablas se pueden crear en cualquier orden.

> **Error frecuente.** Olvidar el ENABLE de una restricción después de una carga de datos. Hasta que lo hagas, la regla no se comprueba y entran filas incorrectas.

Objetos del esquema

Vistas

Una vista es una consulta guardada con nombre. Se consulta como una tabla, pero guarda la consulta, no las filas: cada vez que la lees, la consulta se vuelve a ejecutar sobre los datos actuales.

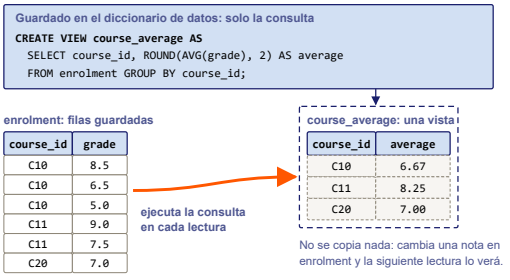
- Una vista con un JOIN, GROUP BY o DISTINCT normalmente no es actualizable directamente.
- `DROP VIEW nombre` elimina la vista; sus tablas y sus filas no se tocan.
- Una **vista materializada** sí guarda filas y se refresca cada cierto tiempo: una fotografía de la habitación, no una ventana.

Oracle · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS product PURGE;
CREATE TABLE product (
  product_id NUMBER(4) PRIMARY KEY,
  name       VARCHAR2(40) NOT NULL,
  price      NUMBER(6,2) NOT NULL,
  on_sale    CHAR(1) DEFAULT 'N' NOT NULL
);
INSERT INTO product VALUES (1, 'Notebook', 3.50, 'Y');
INSERT INTO product VALUES (2, 'Pen', 1.20, 'N');
COMMIT;

-- Una vista actualizable que solo deja trabajar con los
productos en oferta
CREATE OR REPLACE VIEW sale_items AS
  SELECT product_id, name, price, on_sale FROM product WHERE
on_sale = 'Y'
  WITH CHECK OPTION;
UPDATE sale_items SET price = 3.00 WHERE product_id = 1;    --
cambia la tabla
INSERT INTO sale_items VALUES (3, 'Ruler', 2.10, 'Y');       -
- visible a través de la vista: se acepta
INSERT INTO sale_items VALUES (4, 'Glue', 1.80, 'N');        -
- ORA-01402: view WITH CHECK OPTION where-clause violation
SELECT * FROM product;

-- Una vista a través de la cual nadie puede escribir
CREATE OR REPLACE VIEW price_list AS
  SELECT name, price FROM product
  WITH READ ONLY;
UPDATE price_list SET price = 0;                             -
- ORA-42399: cannot perform a DML operation on a read-only
view
```



Una vista es una ventana: guarda la consulta, y las filas salen de la tabla base en cada lectura.

Ejemplo que puedes ejecutar en la web

```
CREATE OR REPLACE VIEW student_grades AS
SELECT s.name, c.title, e.grade
FROM enrolment e
JOIN student s ON s.student_id = e.student_id
JOIN course  c ON c.course_id  = e.course_id;
SELECT * FROM student_grades WHERE grade >= 7 ORDER BY grade DESC;
-- La vista guarda la consulta, no las filas: cambia una nota en la tabla...
UPDATE enrolment SET grade = 9.5 WHERE student_id = 'S03' AND course_id = 'C10';
-- ...y la vista la muestra al instante
SELECT * FROM student_grades WHERE name = 'Eva Sanz';
```

```
CREATE OR REPLACE VIEW course_average AS
SELECT course_id, COUNT(*) AS students, ROUND(AVG(grade), 2) AS average
FROM enrolment
GROUP BY course_id;
SELECT * FROM course_average;
```

> **Ejemplo.** `CREATE OR REPLACE VIEW course_average AS SELECT course_id, ROUND(AVG(grade), 2) AS average FROM enrolment GROUP BY course_id` da a todo el mundo las medias sin enseñar ni una sola nota.

> **Error frecuente.** Pensar que una vista es una copia que hay que refrescar. Una vista normal muestra siempre los datos actuales, y es exactamente igual de rápida que la consulta que lleva dentro.

Secuencias y sinónimos

Una secuencia reparte números únicos, uno tras otro, a cualquier sesión que los pida. Un sinónimo es otro nombre para un objeto, a menudo para una tabla de otro esquema.

- Las secuencias pueden dejar **huecos** (ROLLBACK, números en caché que se pierden al reiniciar la base de datos): nunca las uses para contar filas.
- `CACHE 20` guarda números en memoria para ir más rápido; `NOCYCLE` (por defecto) se detiene en MAXVALUE en lugar de volver a empezar.
- La simulación del navegador no puede ejecutar secuencias ni sinónimos: ejecuta el script en FreeSQL.

¿Secuencia o columna de identidad?

	Secuencia	Columna de identidad
Pertenece a	El esquema: cualquier tabla puede usarla	Una columna de una tabla
Cómo se usa	`seq.NEXTVAL` en el INSERT, o como DEFAULT de la columna	Se deja la columna fuera del INSERT
Numeración compartida	Sí	No
Huecos	Posibles	Posibles (usa una secuencia por dentro)

Oracle · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS ticket PURGE;
DROP SEQUENCE IF EXISTS ticket_seq;
CREATE SEQUENCE ticket_seq START WITH 100 INCREMENT BY 1 NOCACHE;
CREATE TABLE ticket (
  ticket_no NUMBER DEFAULT ticket_seq.NEXTVAL PRIMARY KEY,
  customer  VARCHAR2(30) NOT NULL
);
INSERT INTO ticket (customer) VALUES ('Ana');                                    -- 100, del DEFAULT
INSERT INTO ticket (ticket_no, customer) VALUES (ticket_seq.NEXTVAL, 'Luis');     -- 101
SELECT ticket_seq.CURRVAL AS last_ticket FROM dual;                              -- otra vez 101, en esta sesión
SELECT * FROM ticket;

-- Un sinónimo: un apodo para la tabla
CREATE OR REPLACE SYNONYM queue FOR ticket;
SELECT * FROM queue;
DROP SYNONYM queue;
```

> **Error frecuente.** Esperar una secuencia sin huecos, como números de factura 1, 2, 3… sin que falte ninguno. Un INSERT deshecho con ROLLBACK ya gastó su número; si la ley exige números consecutivos, genéralos de otra forma.

Consultas

SELECT: columnas, expresiones y alias

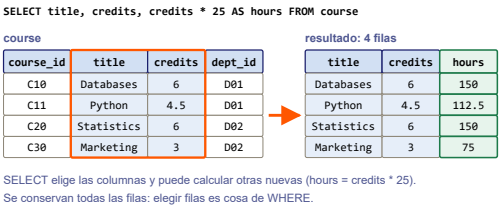
Un SELECT nombra las columnas que quieres ver y FROM la tabla de la que salen. Cada columna puede ser un valor guardado, un cálculo o un texto concatenado, y un alias le da un nombre legible.

- Orden de una consulta básica: `SELECT columnas FROM tabla`; las cláusulas que añadirás después (WHERE, ORDER BY) van detrás de FROM.

- Las comillas simples son para valores de texto; las dobles, para nombres (alias, columnas).
- Un alias solo renombra la salida: no cambia la tabla.

Ejemplo que puedes ejecutar en la web

```
SELECT title, credits, credits * 25 AS hours
FROM course;
SELECT student_id || ' - ' || name AS student_label
FROM student;
SELECT DISTINCT dept_id
FROM course;
```



SELECT elige las columnas y puede calcular otras nuevas; todas las filas de la tabla se mantienen.

Ejemplo. `SELECT title, credits * 25 AS hours FROM course` muestra cada asignatura con sus horas de estudio (1 crédito ECTS ≈ 25 horas).

Error frecuente. Unir texto con `+` (`name + ' ' + email`). En Oracle `+` es solo aritmético y falla con ORA-01722 (número no válido): usa `||`.

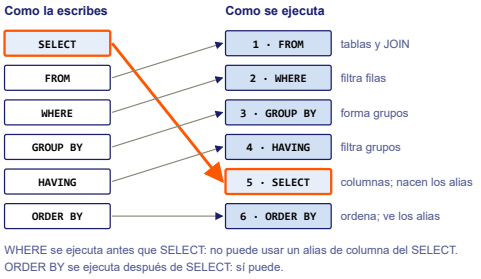
Ordenar resultados con ORDER BY

Sin `ORDER BY` una tabla no tiene un orden garantizado: las filas salen en el orden en que el SGBD las encuentra. `ORDER BY` ordena el resultado por una o más claves, ascendente (`ASC`, por defecto) o descendente (`DESC`).

- ORDER BY es siempre la última cláusula de la consulta (FETCH FIRST va detrás).
- El texto se ordena carácter a carácter: con la ordenación binaria por defecto de Oracle, las mayúsculas van antes que las minúsculas.
- Si dos filas empatan en todas las claves su orden no está definido: añade una clave que lo haga único.

Ejemplo que puedes ejecutar en la web

```
SELECT title, credits
FROM course
ORDER BY credits DESC, title;
SELECT name, email
FROM student
ORDER BY email NULLS FIRST;
SELECT student_id, course_id, grade
FROM enrolment
ORDER BY grade DESC
FETCH FIRST 3 ROWS ONLY;
```



El orden en que escribes las cláusulas no es el orden en que se ejecutan: SELECT va en quinto lugar, así que solo ORDER BY ve sus alias.

Ejemplo. `ORDER BY grade DESC FETCH FIRST 3 ROWS ONLY` da las tres mejores notas.

Error frecuente. Escribir `DESC` una vez y esperar que se aplique a todas las claves. En `ORDER BY credits, title DESC` solo el título es descendente.

Filtrar con WHERE, AND, OR y NOT

`WHERE` conserva solo las filas para las que una condición es verdadera. Las condiciones comparan valores (`=`, `<>`, `<`, `<=`, `>`, `>=`) y se combinan con `AND`, `OR` y `NOT`.

- Las comparaciones de texto distinguen mayúsculas en Oracle: `'spain'` no es `'Spain'`.
- Los números se escriben sin comillas: `credits >= 6`.
- Lee tu WHERE en voz alta con los paréntesis: si la frase no es lo que pide el enunciado, la consulta está mal.

Ejemplo que puedes ejecutar en la web

```
-- Sin paréntesis: AND se evalúa primero
SELECT course_id, title, credits, dept_id
FROM course
WHERE dept_id = 'D01' OR dept_id = 'D02' AND credits >= 6;
-- Con paréntesis: lo que realmente queríamos
SELECT course_id, title, credits, dept_id
FROM course
WHERE (dept_id = 'D01' OR dept_id = 'D02') AND credits >= 6;
```

```
SELECT title
FROM course
WHERE NOT dept_id = 'D01';
```

Primero NOT, luego AND y después OR, como × antes que + en aritmética

1 · Tal como está escrita: AND se evalúa primero

```
dept_id='D01' OR (dept_id='D02' AND credits >= 6)
```

2 · Con los paréntesis que queríamos

```
(dept_id='D01' OR dept_id='D02') AND credits >= 6
```

Ejemplo. `WHERE (dept_id = 'D01' OR dept_id = 'D02') AND credits >= 6` conserva las asignaturas de 6 créditos de cualquiera de los dos departamentos.

course_id	title	credits	dept_id	1	2
C10	Databases	6	D01	✓	✓
C11	Python	4.5	D01	✓	✗
C20	Statistics	6	D02	✓	✓
C30	Marketing	3	D02	✗	✗

Python es de D01: la consulta 1 la deja; tiene 4,5 créditos: la consulta 2 la quita.

Error frecuente. Omitir los paréntesis al mezclar AND y OR. `dept_id = 'D01' OR dept_id = 'D02' AND credits >= 6` devuelve **todas** las asignaturas de D01, tengan los créditos que tengan.

Las mismas tres condiciones, con y sin paréntesis. Solo Python cambia de lado.

BETWEEN, IN y LIKE

Tres atajos para filtros habituales: BETWEEN para rangos, IN para listas de valores y LIKE para patrones de texto.

- `BETWEEN 2020 AND 2021` sobre una columna DATE es un error: compara con fechas, `BETWEEN DATE '2020-01-01' AND DATE '2021-12-31'`.
- Cuidado con los espacios en los patrones: `'The %'` (la palabra "The") no es `'The%'` (que también encuentra "Theo").
- La simulación del navegador ignora las mayúsculas en LIKE; Oracle no. Comprueba en FreeSQL los patrones que dependen de ellas.

Ejemplo que puedes ejecutar en la web

```
SELECT student_id, course_id, grade
FROM enrolment
WHERE grade BETWEEN 7 AND 8.5;
SELECT course_id, title
FROM course
WHERE course_id IN ('C10', 'C30');
SELECT name
FROM student
WHERE name LIKE '%z';
```

Ejemplo. `WHERE grade BETWEEN 7 AND 8.5` devuelve las notas 7.0, 7.5 y 8.5: los dos límites se incluyen.

Error frecuente. Escribir un patrón con `=`: `name = 'A%'` busca un nombre que sea literalmente "A%". Los patrones necesitan LIKE.

Trabajar con NULL

NULL significa "desconocido" o "falta": no es cero ni un texto vacío. Cualquier comparación con NULL es desconocida, así que `= NULL` nunca encuentra nada: usa IS NULL e IS NOT NULL.

- `NOT (email = NULL)` sigue siendo desconocido: negar lo desconocido no lo hace verdadero.
- Una condición como `grade <> 5` también descarta las filas en las que grade es NULL.
- Con ORDER BY, Oracle coloca los NULL al final en orden ascendente (mira la tarjeta de ORDER BY).

Ejemplo que puedes ejecutar en la web

```
-- No devuelve nada: una comparación con NULL nunca es verdadera
SELECT name, email FROM student WHERE email = NULL;
SELECT name FROM student WHERE email IS NULL;
SELECT name, NVL(email, 'sin email') AS email FROM student;
```

WHERE solo conserva una fila si la prueba es VERDADERO

name	email	email = NULL	email IS NULL
Ana Ruiz	ana@uni.es	DESCONOCIDO	FALSO
Luis Gil	luis@uni.es	DESCONOCIDO	FALSO
Eva Sanz	*NULL*	DESCONOCIDO	VERDADERO

= NULL es DESCONOCIDO en todas, incluso en Eva: 0 filas.
IS NULL es la prueba que funciona: devuelve a Eva.
NOT DESCONOCIDO sigue siendo DESCONOCIDO.

V verdadero · F falso · D desconocido

AND	V	D	F
V	V	D	F
D	D	D	F
F	F	F	F

OR	V	D	F
V	V	V	V
D	V	D	D
F	V	D	F

Ejemplo. `SELECT name FROM student WHERE email IS NULL` devuelve los estudiantes sin email.

Comparar con NULL da DESCONOCIDO, y WHERE descarta las filas desconocidas igual que las falsas.

Error frecuente. Escribir `WHERE email = NULL` o `WHERE email = ''`: ninguna devuelve filas. Usa `IS NULL`.

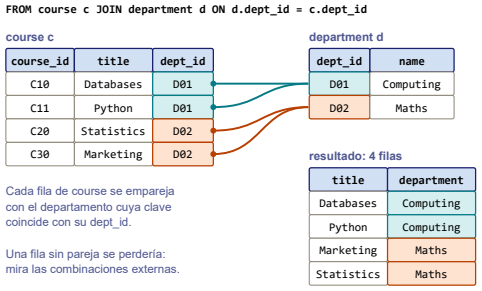
Combinar tablas con INNER JOIN

Un JOIN combina filas de dos tablas relacionadas. La condición ON indica cómo emparejarlas, casi siempre siguiendo una clave ajena hasta la clave primaria a la que apunta.

- Un JOIN por cada clave ajena que sigues: n tablas necesitan n − 1 condiciones de combinación.
- WHERE y ORDER BY van después de todos los JOIN y pueden usar columnas de cualquier tabla combinada.
- Sin la condición ON cada fila se emparejaría con todas las filas de la otra tabla (un producto cartesiano).

Ejemplo que puedes ejecutar en la web

```
SELECT c.title, d.name AS department
FROM course c
JOIN department d ON d.dept_id = c.dept_id
ORDER BY d.name, c.title;
SELECT s.name, c.title, e.grade
FROM enrolment e
JOIN student s ON s.student_id = e.student_id
JOIN course  c ON c.course_id  = e.course_id
WHERE e.grade >= 7
ORDER BY s.name, c.title;
```



La condición ON sigue la clave ajena dept_id hasta el departamento al que referencia.

Mira el vídeo en la web, en la sección SQL.

> **Ejemplo.** `SELECT s.name, c.title FROM enrolment e JOIN student s ON s.student_id = e.student_id JOIN course c ON c.course_id = e.course_id` lista quién está matriculado en qué.

> **Error frecuente.** Combinar por columnas equivocadas (`ON c.course_id = d.dept_id`). La consulta se ejecuta, pero las filas no tienen sentido o faltan: el ON debe seguir una clave ajena.

Combinar tablas

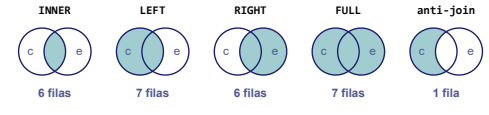
Combinaciones externas: LEFT, RIGHT y FULL

Un OUTER JOIN conserva también las filas que no encuentran pareja y rellena con NULL el lado que falta. LEFT conserva todas las filas de la tabla de la izquierda, RIGHT todas las de la derecha y FULL todas las de ambas.

- Una condición sobre la tabla derecha va en el **ON** si quieres conservar las filas sin pareja; en el WHERE las vuelve a eliminar.
- Para contar parejas, cuenta una columna de la tabla derecha (`COUNT(e.course_id)`): una asignatura sin pareja sigue produciendo una fila, y `COUNT(*)` la contaría como 1.
- La antigua sintaxis `(+)` de Oracle (`WHERE c.course_id = e.course_id(+)`) es un LEFT JOIN: reconócela en código antiguo, pero escribe la forma ANSI.

Ejemplo que puedes ejecutar en la web

```
-- INNER: Marketing (sin matrículas) desaparece
SELECT c.title, e.student_id, e.grade
FROM course c
JOIN enrolment e ON e.course_id = c.course_id
ORDER BY c.title;
-- LEFT: todas las asignaturas, con NULL donde no hay nadie
matriculado
SELECT c.title, e.student_id, e.grade
FROM course c
LEFT JOIN enrolment e ON e.course_id = c.course_id
ORDER BY c.title;
-- Anticombinación: las asignaturas que no cursa nadie
SELECT c.course_id, c.title
FROM course c
LEFT JOIN enrolment e ON e.course_id = c.course_id
WHERE e.course_id IS NULL;
-- FULL: quién cursa Statistics (C20), quién cursa Python
(C11) y quién cursa las dos
SELECT st.student_id AS in_statistics, py.student_id AS
in_python
```





Sombreado: las filas que conserva cada combinación, contadas en las tablas de ejemplo. Solo Marketing se queda sin pareja.

```
FROM (SELECT student_id FROM enrolment WHERE course_id = 'C20')
FULL OUTER JOIN (SELECT student_id FROM enrolment WHERE course_i
  ON py.student_id = st.student_id;
```

> **Ejemplo.** `SELECT c.title FROM course c LEFT JOIN enrolment e ON e.course_id = c.course_id WHERE e.course_id IS NULL` lista las asignaturas en las que no se ha matriculado nadie: Marketing.

> **Error frecuente.** Filtrar la tabla derecha en el WHERE: `LEFT JOIN enrolment e ON … WHERE e.grade >= 5` vuelve a eliminar las asignaturas sin matrículas, porque su nota es NULL. Lleva la condición al ON: `ON e.course_id = c.course_id AND e.grade >= 5`.

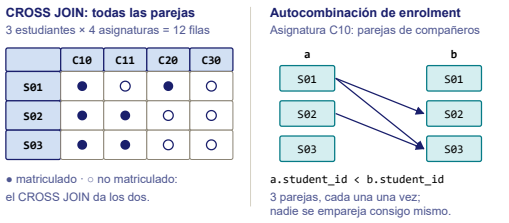
CROSS JOIN, autocombinaciones, USING y NATURAL

Un CROSS JOIN empareja cada fila con todas las filas de otra tabla. Una autocombinación (self-join) combina una tabla consigo misma con dos alias. USING y NATURAL JOIN son atajos para combinar por columnas que se llaman igual.

- n filas × m filas: un CROSS JOIN crece muy deprisa; 1.000 × 1.000 es un millón de filas.
- Olvidar la condición de combinación con la sintaxis antigua (`FROM a, b`) produce un CROSS JOIN accidental.
- Prefiere `ON` (o `USING`) a NATURAL JOIN: la condición de combinación queda escrita a la vista de todos.

Ejemplo que puedes ejecutar en la web

```
SELECT s.name, c.title
FROM student s
CROSS JOIN course c
ORDER BY s.name, c.title;
-- Autocombinación: parejas de compañeros en la misma
asignatura
SELECT a.course_id, a.student_id AS student_1, b.student_id AS
student_2
FROM enrolment a
JOIN enrolment b ON b.course_id = a.course_id AND a.student_id
< b.student_id
ORDER BY a.course_id;
SELECT dept_id, d.name AS department, c.title
FROM course c
JOIN department d USING (dept_id);
SELECT title, name FROM course NATURAL JOIN department;
-- Una columna nueva con un nombre común cambia el NATURAL
JOIN
ALTER TABLE course ADD (name VARCHAR2(30));
SELECT title, name FROM course NATURAL JOIN department;
```



Un CROSS JOIN enumera todas las combinaciones; una autocombinación empareja filas de una misma tabla con dos alias.

> **Ejemplo.** Cada estudiante con cada asignatura: `SELECT s.name, c.title FROM student s CROSS JOIN course c` (3 × 4 = 12 filas).

> **Error frecuente.** Confiar en NATURAL JOIN en código que tiene que durar. En el ejemplo, añadir a `course` una columna inofensiva llamada `name` hace que `course NATURAL JOIN department` combine por `dept_id` **y** por `name`, y deje de devolver filas sin avisar.

Agregación

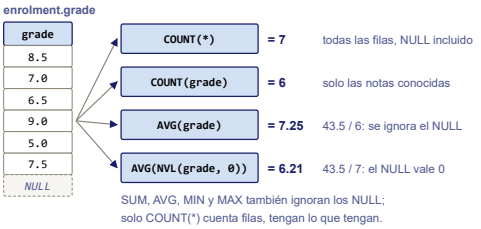
Funciones de agregado: COUNT, SUM, AVG, MIN y MAX

Una función de agregado convierte muchas filas en un solo valor: cuántas hay, el total, la media, la menor, la mayor. Sin GROUP BY, todas las filas (tras el WHERE) forman un único grupo y el resultado es una sola fila.

- MIN y MAX funcionan también con texto (orden alfabético) y fechas (la más antigua, la más reciente); SUM y AVG necesitan números.
- Sobre un conjunto vacío COUNT devuelve 0, pero SUM, AVG, MIN y MAX devuelven NULL.
- No se puede mezclar un agregado con una columna normal sin GROUP BY: `SELECT name, COUNT(*) FROM student` falla con ORA-00937 (not a single-group group function).

Ejemplo que puedes ejecutar en la web

```
SELECT COUNT(*) AS students, COUNT(email) AS with_email
FROM student;
SELECT COUNT(*) AS enrolments, COUNT(DISTINCT course_id) AS
courses_taken,
       MIN(grade) AS lowest, MAX(grade) AS highest,
ROUND(AVG(grade), 2) AS average
FROM enrolment;
-- Una nota que falta se ignora, no cuenta como cero
INSERT INTO enrolment VALUES ('S01', 'C11', NULL);
SELECT COUNT(*) AS enrolments, COUNT(grade) AS graded,
       ROUND(AVG(grade), 2) AS avg_known, ROUND(AVG(NVL(grade,
0)), 2) AS avg_as_zero
FROM enrolment;
```



Los agregados ignoran los NULL; solo COUNT(*) cuenta todas las filas.

Ejemplo. `SELECT COUNT(*) AS enrolments, ROUND(AVG(grade), 2) AS average, MAX(grade) AS best FROM enrolment` resume toda la tabla en una fila.

Error frecuente. Usar `COUNT(*)` para contar valores que pueden faltar. Para saber cuántos estudiantes tienen email, cuenta la columna: `COUNT(email)`.

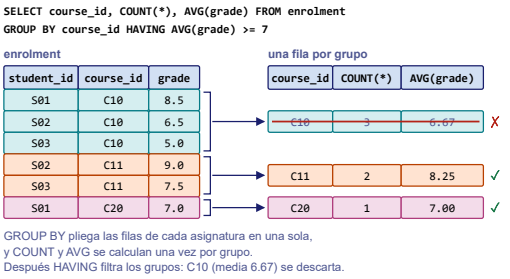
GROUP BY y HAVING

GROUP BY reparte las filas en grupos que comparten un valor y calcula los agregados una vez por grupo. HAVING filtra los grupos, igual que WHERE filtra las filas.

- WHERE filtra filas **antes** de agrupar (`WHERE grade >= 6`: solo cuentan esas notas); HAVING filtra grupos **después** (`HAVING COUNT(*) >= 2`: solo los grupos con dos filas o más).
- Se puede agrupar por varias columnas: un grupo por cada combinación distinta de valores.
- Una condición sin agregado va en WHERE: queda más clara y descarta las filas antes.

Ejemplo que puedes ejecutar en la web

```
SELECT course_id, COUNT(*) AS students, ROUND(AVG(grade), 2)
AS average
FROM enrolment
GROUP BY course_id
ORDER BY course_id;
-- Por departamento: primero el JOIN, luego agrupar por una
columna de la tabla combinada
SELECT d.name AS department, COUNT(*) AS courses,
SUM(c.credits) AS credits
FROM course c
JOIN department d ON d.dept_id = c.dept_id
GROUP BY d.name;
-- WHERE (filas) y HAVING (grupos) a la vez
SELECT course_id, COUNT(*) AS grades_6_or_more
FROM enrolment
WHERE grade >= 6
GROUP BY course_id
HAVING COUNT(*) >= 2
ORDER BY course_id;
```



Un montón por asignatura y una fila de resultado por montón; después HAVING descarta montones enteros.

Mira el vídeo en la web, en la sección SQL.

Ejemplo. `SELECT course_id, ROUND(AVG(grade), 2) FROM enrolment GROUP BY course_id HAVING AVG(grade) >= 7` conserva las asignaturas cuya media es al menos 7.

Error frecuente. Escribir `WHERE AVG(grade) > 7`, que falla con ORA-00934 (group function is not allowed here). Las condiciones sobre agregados van en HAVING.

Subconsultas y conjuntos

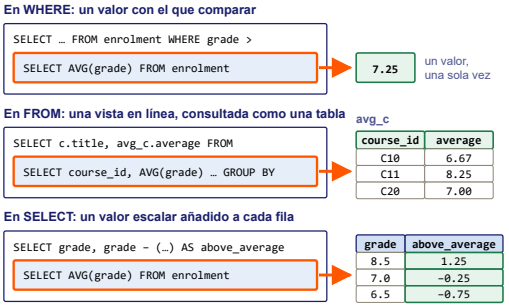
Subconsultas en WHERE, FROM y SELECT

Una subconsulta es un SELECT dentro de otra sentencia, entre paréntesis. Responde a una pregunta que la consulta exterior necesita antes: «¿cuál es la nota media?» y, después, «¿quién está por encima?».

- Si una subconsulta de una fila devuelve varias, Oracle falla con ORA-01427 (single-row subquery returns more than one row).
- Si no devuelve ninguna fila, su valor es NULL, y una comparación con NULL no conserva nada.
- Una subconsulta escalar en el SELECT debe devolver como mucho una fila y una columna.

Ejemplo que puedes ejecutar en la web

```
-- En WHERE: las notas por encima de la media global
SELECT student_id, course_id, grade
FROM enrolment
WHERE grade > (SELECT AVG(grade) FROM enrolment)
ORDER BY grade DESC;
-- En FROM (vista en línea): la media por asignatura, luego
filtrada y combinada
SELECT c.title, avg_c.average
FROM (SELECT course_id, ROUND(AVG(grade), 2) AS average
      FROM enrolment
      GROUP BY course_id) avg_c
JOIN course c ON c.course_id = avg_c.course_id
WHERE avg_c.average >= 7;
-- En SELECT (escalar): cada nota comparada con la media
global
SELECT student_id, course_id, grade,
       grade - (SELECT AVG(grade) FROM enrolment) AS
above_average
FROM enrolment;
```



La consulta interior resaltada se ejecuta primero; el sitio donde está decide para qué se usa su resultado.

Mira el vídeo en la web, en la sección SQL.

> **Ejemplo.** `SELECT title FROM course WHERE credits = (SELECT MAX(credits) FROM course)` lista las asignaturas con más créditos.

> **Error frecuente.** Escribir `WHERE grade > AVG(grade)`. Un agregado no puede ir en el WHERE; calcúlalo en una subconsulta: `WHERE grade > (SELECT AVG(grade) FROM enrolment)`.

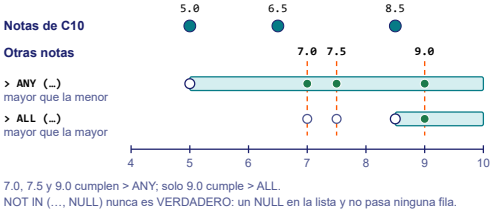
Subconsultas de varias filas: IN, ANY y ALL

Cuando una subconsulta devuelve una lista de valores, se compara con IN (igual a uno de ellos), ANY (cierto para al menos uno) o ALL (cierto para todos). `NOT IN` esconde una trampa: basta un NULL en la lista para que no salga ninguna fila.

- Una subconsulta tras = que devuelve varias filas falla (ORA-01427); usa IN.
- `> ALL` con una lista vacía es cierto para todas las filas; `> ANY` con una lista vacía es falso.
- IN con una subconsulta suele leerse mejor que un JOIN cuando solo necesitas columnas de la tabla exterior.

Ejemplo que puedes ejecutar en la web

```
SELECT name
FROM student
WHERE student_id IN (SELECT student_id FROM enrolment WHERE
course_id = 'C11');
-- Mejor que todas las notas de Databases (C10)
SELECT student_id, course_id, grade
FROM enrolment
WHERE grade > ALL (SELECT grade FROM enrolment WHERE course_id
= 'C10');
-- Mejor que al menos una nota de Databases
SELECT student_id, course_id, grade
FROM enrolment
WHERE course_id <> 'C10'
```



`> ANY` significa por encima del menor valor de la lista; `> ALL`, por encima del mayor.

```
  AND grade > ANY (SELECT grade FROM enrolment WHERE course_id =
-- La trampa de NOT IN: las notas que nadie ha sacado en Python
SELECT student_id, course_id, grade
FROM enrolment
WHERE grade NOT IN (SELECT grade FROM enrolment WHERE course_id
-- ...dejan de funcionar en cuanto una nota de Python es NULL
INSERT INTO enrolment VALUES ('S01', 'C11', NULL);
SELECT student_id, course_id, grade
FROM enrolment
WHERE grade NOT IN (SELECT grade FROM enrolment WHERE course_id
```

Ejemplo. Los estudiantes que cursan alguna asignatura de menos de 5 créditos: `WHERE student_id IN (SELECT e.student_id FROM enrolment e JOIN course c ON c.course_id = e.course_id WHERE c.credits < 5)`.

Error frecuente. Usar NOT IN con una subconsulta que puede devolver NULL. En el ejemplo basta una matrícula de Python sin nota para que «las notas que nadie ha sacado en Python» no devuelvan ninguna fila.

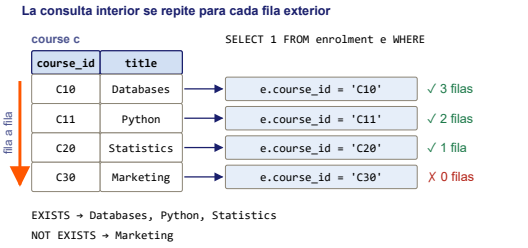
Subconsultas correlacionadas y EXISTS

Una subconsulta correlacionada usa una columna de la consulta exterior, así que se vuelve a evaluar para cada fila exterior. EXISTS pregunta si una subconsulta devuelve al menos una fila; NOT EXISTS, si no devuelve ninguna.

- Da a la tabla interior su propio alias (`x`), para que quede claro qué columnas son de la fila exterior (`e`).
- EXISTS puede parar en la primera coincidencia, y el optimizador suele convertir EXISTS e IN en combinaciones de todos modos.
- NOT EXISTS es la forma segura de escribir «no tiene ningún …»; NOT IN falla con los NULL.

Ejemplo que puedes ejecutar en la web

```
-- Quién ha superado la media de su propia asignatura
SELECT e.student_id, e.course_id, e.grade
FROM enrolment e
WHERE e.grade > (SELECT AVG(x.grade) FROM enrolment x WHERE
x.course_id = e.course_id)
ORDER BY e.course_id;
-- Asignaturas con al menos una matrícula, y asignaturas sin
ninguna
SELECT c.title FROM course c
WHERE EXISTS (SELECT 1 FROM enrolment e WHERE e.course_id =
c.course_id);
SELECT c.title FROM course c
WHERE NOT EXISTS (SELECT 1 FROM enrolment e WHERE e.course_id
= c.course_id);
-- El primero de cada grupo: la mejor nota de cada asignatura
SELECT e.course_id, e.student_id, e.grade
FROM enrolment e
WHERE e.grade = (SELECT MAX(x.grade) FROM enrolment x WHERE
x.course_id = e.course_id)
ORDER BY e.course_id;
```



La consulta interior se vuelve a ejecutar con el id de cada fila exterior.

Ejemplo. Los estudiantes no matriculados en Statistics: `SELECT name FROM student s WHERE NOT EXISTS (SELECT 1 FROM enrolment e WHERE e.student_id = s.student_id AND e.course_id = 'C20')`.

Error frecuente. Olvidar la condición de correlación (`e.course_id = c.course_id`). La subconsulta ignora entonces la fila exterior, y EXISTS es cierto (o falso) para todas las filas a la vez.

Operadores de conjuntos: UNION, INTERSECT y MINUS

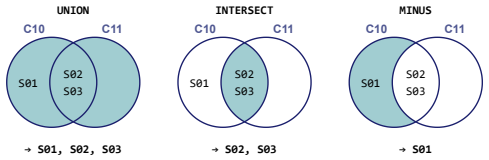
Los operadores de conjuntos combinan los resultados de dos consultas: UNION (filas de cualquiera de las dos), INTERSECT (filas de ambas) y MINUS (filas de la primera que no están en la segunda). UNION ALL conserva los duplicados.

- Oracle llama **MINUS** a la diferencia; el estándar y otros SGBD la llaman EXCEPT (las versiones recientes de Oracle aceptan las dos, y MINUS funciona en todas).

- Los operadores de conjuntos comparan filas completas, y aquí dos NULL cuentan como iguales.
- Para poner columnas de dos tablas una al lado de otra hace falta un JOIN; los operadores de conjuntos apilan filas.

Ejemplo que puedes ejecutar en la web

```
-- Estudiantes de Databases (C10) y de Python (C11)
SELECT student_id FROM enrolment WHERE course_id = 'C10'
UNION
SELECT student_id FROM enrolment WHERE course_id = 'C11';
SELECT student_id FROM enrolment WHERE course_id = 'C10'
UNION ALL
SELECT student_id FROM enrolment WHERE course_id = 'C11';
SELECT student_id FROM enrolment WHERE course_id = 'C10'
INTERSECT
SELECT student_id FROM enrolment WHERE course_id = 'C11';
SELECT student_id FROM enrolment WHERE course_id = 'C10'
MINUS
SELECT student_id FROM enrolment WHERE course_id = 'C11';
-- Una sola lista de nombres de dos tablas, ordenada una vez
al final
SELECT name, 'student' AS kind FROM student
UNION
SELECT name, 'department' FROM department
ORDER BY name;
```



C10: estudiantes de Databases · C11: estudiantes de Python
UNION ALL conserva los repetidos: 3 + 2 = 5 filas.

Los operadores de conjuntos comparan filas completas del resultado: aquí, los id de los estudiantes de dos asignaturas.

Ejemplo. Los estudiantes de Databases o de Statistics: `SELECT student_id FROM enrolment WHERE course_id = 'C10' UNION SELECT student_id FROM enrolment WHERE course_id = 'C20'`.

Error frecuente. Escribir un ORDER BY dentro de la primera consulta de un UNION. Oracle falla con ORA-00933 (SQL command not properly ended): ordena una sola vez, tras la última consulta.

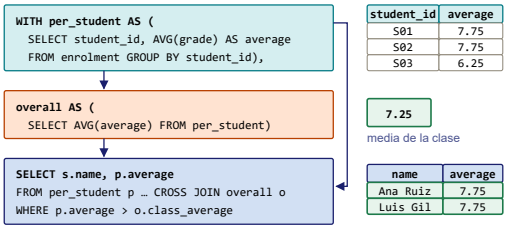
WITH: expresiones de tabla comunes

La cláusula WITH da nombre a una subconsulta al principio de una sentencia, una expresión de tabla común (CTE), y la consulta principal la usa como una tabla. Se pueden encadenar varias CTE, cada una usando las anteriores.

- Un solo WITH por sentencia, al principio; las CTE se separan con comas y el SELECT principal va al final.
- Una CTE se puede usar varias veces en la consulta principal, en lugar de repetir la misma subconsulta.
- Una CTE que se refiere a sí misma (recursiva) recorre jerarquías, como un organigrama.

Ejemplo que puedes ejecutar en la web

```
WITH course_avg AS (
  SELECT course_id, ROUND(AVG(grade), 2) AS average
  FROM enrolment
  GROUP BY course_id
)
SELECT c.title, ca.average
FROM course_avg ca
JOIN course c ON c.course_id = ca.course_id
ORDER BY ca.average DESC;
-- CTE encadenadas: cada paso usa el anterior
WITH per_student AS (
  SELECT student_id, AVG(grade) AS average
  FROM enrolment
  GROUP BY student_id
),
overall AS (
  SELECT AVG(average) AS class_average FROM per_student
)
SELECT s.name, ROUND(p.average, 2) AS average,
ROUND(o.class_average, 2) AS class_average
FROM per_student p
JOIN student s ON s.student_id = p.student_id
```



Cada paso con nombre puede usar los anteriores; el SELECT final usa los dos.

Cada CTE es un paso con nombre; los pasos siguientes y el SELECT final lo usan como una tabla.

```
CROSS JOIN overall o
WHERE p.average > o.class_average;
```

> **Ejemplo.** Los estudiantes cuya media supera la media de la clase: una CTE con la media de cada estudiante, otra con la media global y un SELECT final que las compara.

> **Error frecuente.** Escribir un punto y coma o un segundo WITH entre las CTE. Forman una sola sentencia: `WITH a AS (…), b AS (…) SELECT …`.

Funciones

Funciones de texto

Las funciones de una fila transforman un valor por fila. Las de texto cambian mayúsculas y minúsculas, recortan, buscan, miden, quitan espacios, rellenan y sustituyen cadenas; `||` las concatena.

- Las funciones se pueden anidar; la más interior se ejecuta primero: `UPPER(SUBSTR(name, 1, 1))`.
- En Oracle `LENGTH('')` es NULL, no 0, porque la cadena vacía es NULL.
- Una función sobre una columna en el WHERE puede impedir que se use un índice (mira las tarjetas de Eficiencia).

Ejemplo que puedes ejecutar en la web

```
SELECT name,
       UPPER(name) AS shouting,
       LOWER(name) AS quiet,
       LENGTH(name) AS letters,
       SUBSTR(name, 1, INSTR(name, ' ') - 1) AS first_name
FROM student;
-- Separar un correo en usuario y dominio
SELECT email,
       INITCAP(SUBSTR(email, 1, INSTR(email, '@') - 1)) AS user_part,
       SUBSTR(email, INSTR(email, '@') + 1) AS domain,
       REPLACE(email, 'uni.es', 'cunef.edu') AS new_email
FROM student
WHERE email IS NOT NULL;
-- Rellenar y concatenar: un índice con puntos guía
SELECT LPAD(SUBSTR(course_id, 2), 4, '0') AS code,
       RPAD(title, 14, '.') || LPAD(credits, 4, '.') AS line
FROM course;
SELECT '[' || TRIM('   Databases   ') || ']' AS trimmed FROM dual;
```

> **Ejemplo.** `SUBSTR(email, INSTR(email, '@') + 1)` devuelve el dominio de un correo: `uni.es`.

> **Error frecuente.** Contar las posiciones desde 0. En Oracle el primer carácter es la posición 1: `SUBSTR('Oracle', 1, 3)` es `Ora`.

Funciones numéricas y de fecha

ROUND, TRUNC, MOD, CEIL y FLOOR dan forma a los números. SYSDATE, la aritmética de fechas, ADD_MONTHS, MONTHS_BETWEEN y EXTRACT trabajan con fechas.

- `ADD_MONTHS(DATE '2026-01-31', 1)` es el 28 de febrero: Oracle te mantiene en el final de mes.
- `MONTHS_BETWEEN` devuelve una fracción; envuélvelo en TRUNC o ROUND para tener meses enteros.
- Los literales de fecha se escriben `DATE '2026-09-14'`, siempre año-mes-día.

Ejemplo que puedes ejecutar en la web

```
SELECT ROUND(7.456, 2) AS round_2, TRUNC(7.456, 2) AS trunc_2, ROUND(7.5) AS round_0,
       CEIL(4.1) AS up, FLOOR(4.9) AS down, MOD(17, 5) AS remainder
FROM dual;
-- Semanas completas de estudio a 10 horas por semana (1 crédito ECTS = 25 horas)
SELECT title, credits * 25 AS hours, CEIL(credits * 25 / 10) AS weeks_at_10h
FROM course;
SELECT DATE '2026-09-14' + 7 AS one_week_later,
       DATE '2027-01-22' - DATE '2026-09-14' AS semester_days,
```

```
       ADD_MONTHS(DATE '2026-01-31', 1) AS end_of_february,
       TRUNC(MONTHS_BETWEEN(DATE '2027-06-30', DATE '2026-09-14')) AS whole_months,
       EXTRACT(YEAR FROM DATE '2026-09-14') AS year_part
FROM dual;
SELECT TRUNC(SYSDATE) AS today, TRUNC(SYSDATE, 'MM') AS first_of_month,
       EXTRACT(MONTH FROM SYSDATE) AS month_number
FROM dual;
```

Ejemplo. Los días que quedan para el final del cuatrimestre: `DATE '2027-01-22' - TRUNC(SYSDATE)`.

Error frecuente. Sumar 30 días para «pasar al mes siguiente». Los meses tienen de 28 a 31 días: usa `ADD_MONTHS`.

Conversión, funciones para NULL, CASE y DECODE

TO_CHAR, TO_DATE y TO_NUMBER convierten entre texto, fechas y números con un modelo de formato. NVL, NVL2 y COALESCE sustituyen los NULL, y CASE (o DECODE, propio de Oracle) elige un valor según condiciones.

- Para comparar fechas usa `TO_DATE` o un literal `DATE '…'`; no confíes nunca en una conversión implícita de texto a fecha.
- Un CASE sin ELSE que no encaja con nada devuelve NULL.
- Todas las ramas de un CASE deben devolver el mismo tipo.

Ejemplo que puedes ejecutar en la web

```
SELECT TO_CHAR(TO_DATE('15/09/2026', 'DD/MM/YYYY'), 'YYYY-MM-DD') AS iso_date,
       TO_CHAR(DATE '2026-09-14', 'DD/MM/YYYY') AS spanish_date,
       TO_NUMBER('12.5') * 2 AS doubled,
       TO_CHAR(8.5, 'FM990.00') AS two_decimals
FROM dual;
SELECT name,
       NVL(email, 'no email') AS email,
       NVL2(email, 'yes', 'no') AS has_email,
       COALESCE(email, name, 'unknown') AS contact
FROM student;
SELECT student_id, course_id, grade,
       CASE
         WHEN grade >= 9 THEN 'A'
         WHEN grade >= 7 THEN 'B'
         WHEN grade >= 5 THEN 'C'
         ELSE 'F'
       END AS letter,
       DECODE(course_id, 'C10', 'Databases', 'C11', 'Python', 'other') AS course_name
FROM enrolment
ORDER BY grade DESC;
```

Ejemplo. `CASE WHEN grade >= 5 THEN 'pass' ELSE 'fail' END` convierte cada nota en un resultado que entiende cualquiera.

Error frecuente. Ordenar mal las cláusulas WHEN. `WHEN grade >= 5 THEN 'pass' WHEN grade >= 9 THEN 'distinction'` nunca dice distinction, porque gana la primera rama cierta: pon primero la condición más estricta.

Modificar datos

DML: leer y modificar filas

SELECT lee, INSERT añade filas, UPDATE las modifica y DELETE las elimina. UPDATE y DELETE afectan a todas las filas que cumplen el WHERE.

- Las combinaciones (JOIN) siguen las claves ajenas: `enrolment.course_id = course.course_id`.
- El ejecutor muestra cuántas filas ha cambiado cada INSERT, UPDATE o DELETE.
- Para quedarte con las primeras filas de un resultado usa `FETCH FIRST n ROWS ONLY`; Oracle no tiene `LIMIT`.

Ejemplo que puedes ejecutar en la web

```
INSERT INTO enrolment VALUES ('S03', 'C20', 6.0);
UPDATE enrolment SET grade = grade + 0.5 WHERE course_id = 'C10' AND grade < 6;
DELETE FROM enrolment WHERE student_id = 'S02' AND course_id = 'C11';
SELECT s.name, c.title, e.grade
FROM enrolment e
JOIN student s ON s.student_id = e.student_id
JOIN course  c ON c.course_id  = e.course_id
ORDER BY s.name, c.title;
SELECT c.title, COUNT(*) AS students, ROUND(AVG(e.grade), 2) AS average
FROM enrolment e JOIN course c ON c.course_id = e.course_id
GROUP BY c.title
HAVING COUNT(*) >= 2;
```

Ejemplo. La nota media por asignatura es `SELECT course_id, AVG(grade) FROM enrolment GROUP BY course_id`.

Error frecuente. Filtrar un agregado con WHERE (`WHERE AVG(grade) > 7`). WHERE se aplica antes de agrupar; usa `HAVING AVG(grade) > 7`.

INSERT en la práctica

INSERT añade filas. Nombra las columnas que rellenas, deja que las columnas con DEFAULT e identidad se rellenen solas, añade varias filas en una sentencia o copia las filas de una consulta con INSERT ... SELECT.

- El texto y las fechas van entre comillas, los números sin ellas: `'S04'`, `DATE '2026-09-14'`, `6.5`.
- Un INSERT que incumple una restricción se rechaza entero: si lleva varias filas, no se inserta ninguna.
- Las filas nuevas no son permanentes hasta el COMMIT.

Ejemplo que puedes ejecutar en la web

```
INSERT INTO student (student_id, name) VALUES ('S04', 'Marta Paz');
SELECT * FROM student;
-- Varias filas en una sentencia (Oracle 23ai)
INSERT INTO course (course_id, title, credits, dept_id) VALUES
  ('C40', 'Cloud computing', 6, 'D01'),
  ('C41', 'Ethics', 3, 'D02');
SELECT * FROM course;
-- INSERT ... SELECT: la columna de identidad y el DEFAULT se rellenan solos
CREATE TABLE honours (
  honours_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  student_id VARCHAR2(3) NOT NULL,
  course_id  VARCHAR2(3) NOT NULL,
  awarded    DATE DEFAULT SYSDATE NOT NULL
);
INSERT INTO honours (student_id, course_id)
SELECT student_id, course_id FROM enrolment WHERE grade >= 8.5;
SELECT * FROM honours;
```

Ejemplo. El cuadro de honor: `INSERT INTO honours (student_id, course_id) SELECT student_id, course_id FROM enrolment WHERE grade >= 8.5` copia los mejores resultados; el identificador y la fecha se rellenan solos.

Error frecuente. Omitir la lista de columnas: `INSERT INTO student VALUES ('S04', 'Marta Paz')` falla con ORA-00947 (not enough values), y todo INSERT escrito así se rompe cuando la tabla gana una columna.

UPDATE y DELETE en la práctica

UPDATE cambia columnas de las filas que cumplen su WHERE, con valores que pueden ser expresiones o subconsultas. DELETE elimina las filas que lo cumplen. Antes de ejecutar cualquiera de los dos, ejecuta su WHERE como un SELECT.

- El ejecutor muestra cuántas filas ha cambiado cada sentencia: compáralo con las filas que devolvió el SELECT.
- `LEAST(grade + 0.5, 10)` sube una nota sin pasar del máximo; `GREATEST` hace lo contrario.
- Hasta el COMMIT, ROLLBACK todavía puede deshacer un UPDATE o un DELETE equivocado.

Ejemplo que puedes ejecutar en la web

```
-- 1. Ejecuta el WHERE como SELECT: ¿qué filas cambiarán?
SELECT * FROM enrolment
WHERE course_id IN (SELECT course_id FROM course WHERE dept_id = 'D01')
  AND grade < 7;
-- 2. El mismo WHERE en el UPDATE: +0,5, nunca por encima de 10
UPDATE enrolment
SET grade = LEAST(grade + 0.5, 10)
WHERE course_id IN (SELECT course_id FROM course WHERE dept_id = 'D01')
  AND grade < 7;
-- Un valor sacado de una subconsulta: Marketing pasa a tener los créditos de Databases
UPDATE course
SET credits = (SELECT credits FROM course WHERE course_id = 'C10')
WHERE course_id = 'C30';
-- DELETE con una subconsulta: las matrículas de todas las asignaturas de Maths
DELETE FROM enrolment
WHERE course_id IN (SELECT course_id FROM course WHERE dept_id = 'D02');
SELECT * FROM enrolment ORDER BY student_id, course_id;
SELECT course_id, title, credits FROM course;
-- ¿No te convence? Deshazlo todo
ROLLBACK;
```

Ejemplo. `UPDATE course SET credits = credits * 1.5 WHERE dept_id = 'D02'` multiplica por 1,5 los créditos de todas las asignaturas de Maths.

Error frecuente. Ejecutar un UPDATE o un DELETE sin WHERE, o con un WHERE que nunca has probado. `UPDATE enrolment SET grade = 10` pone un 10 a todo el mundo.

DCL: usuarios y privilegios

DCL: usuarios, roles y privilegios

GRANT concede un privilegio sobre un objeto a un usuario o rol; REVOKE lo retira. Da a cada rol solo lo que necesita: el principio de mínimo privilegio.

- `WITH GRANT OPTION` permite a quien lo recibe conceder ese mismo privilegio a otros; úsalo con moderación.
- REVOKE retira el privilegio a ese rol; sus miembros lo pierden salvo que otro rol se lo siga concediendo.
- Oracle nombra los objetos como `esquema.tabla`: `GRANT SELECT ON hr.enrolment TO teacher` se refiere a la tabla del esquema `hr`.

Oracle · no se ejecuta aquí

```
CREATE ROLE teacher;
CREATE ROLE reporting;
CREATE USER ana IDENTIFIED BY "change-me";
GRANT CREATE SESSION TO ana;
GRANT teacher TO ana;

GRANT SELECT, INSERT, UPDATE ON enrolment TO teacher;
GRANT SELECT ON student TO teacher;
GRANT SELECT ON course  TO teacher;

CREATE VIEW course_average AS
  SELECT course_id, AVG(grade) AS average FROM enrolment GROUP BY course_id;
GRANT SELECT ON course_average TO reporting;      -- sin acceso a las notas individuales

REVOKE UPDATE ON enrolment FROM teacher;
```

Error frecuente. Dejar que la aplicación se conecte como propietaria de la base de datos. Un fallo o una inyección SQL puede entonces borrar tablas; con un rol limitado a DML sobre sus tablas, no puede.

Privilegios de sistema, privilegios de objeto y roles

Oracle tiene dos clases de privilegio. Un privilegio de sistema permite una clase de acción (`CREATE SESSION`, `CREATE TABLE`); un privilegio de objeto permite una acción sobre un objeto concreto (`SELECT ON hr.emp`). Un rol agrupa privilegios para concederlos de una vez.

- Concede los privilegios a roles, no a personas: cuando alguien cambia de puesto cambias una pertenencia a un rol en lugar de veinte GRANT.
- Los privilegios recibidos a través de un rol no cuentan dentro de un procedimiento almacenado que se ejecuta con los derechos de su propietario: concédelos directamente al propietario.
- En FreeSQL trabajas en un esquema sin derechos de administrador: puedes conceder privilegios de objeto sobre tus tablas, pero crear usuarios y roles exige un administrador.

¿Privilegio de sistema o de objeto?

	Privilegio de sistema	Privilegio de objeto
Permite	Una clase de acción	Una acción sobre un objeto concreto
Ejemplos	`CREATE SESSION`, `CREATE TABLE`, `CREATE VIEW`, `CREATE SEQUENCE`, `CREATE PROCEDURE`	`SELECT`, `INSERT`, `UPDATE`, `DELETE` sobre una tabla; `EXECUTE` sobre un procedimiento
Sintaxis	`GRANT CREATE TABLE TO ana`	`GRANT SELECT ON hr.emp TO ana`
Para pasarlo	`WITH ADMIN OPTION`	`WITH GRANT OPTION`
REVOKE a quien lo pasó	No se propaga	Se propaga a todos los que lo recibieron por esa vía
Lo concede	Un administrador, o quien lo tenga WITH ADMIN OPTION	El propietario, o quien lo tenga WITH GRANT OPTION

Oracle · necesita una cuenta de administrador

```
CREATE USER app_user IDENTIFIED BY "Change-me-1"
  DEFAULT TABLESPACE users QUOTA 10M ON users;
GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW TO app_user;      -- privilegios de sistema

CREATE ROLE jefe;                                                  -- un conjunto de privilegios
GRANT SELECT, INSERT, UPDATE, DELETE ON hr.dept TO jefe;           -- privilegios de objeto
GRANT CREATE SESSION TO jefe;
GRANT jefe TO app_user;

GRANT SELECT ON hr.emp TO app_user WITH GRANT OPTION;              -- app_user puede pasarlo
GRANT CREATE SEQUENCE TO app_user WITH ADMIN OPTION;               -- app_user puede pasarlo

-- Conectado como app_user: ¿qué puedo usar ahora?
SELECT * FROM session_privs ORDER BY privilege;
SELECT * FROM session_roles;

-- Retirarlos
REVOKE SELECT ON hr.emp FROM app_user;      -- también a todos los que lo recibieron de app_user
REVOKE CREATE SEQUENCE FROM app_user;       -- quienes lo recibieron de app_user lo conservan
REVOKE jefe FROM app_user;
DROP ROLE jefe;
```

> **Ejemplo.** Un usuario de informes recibe `CREATE SESSION` (de sistema) y `SELECT ON hr.course_average` (de objeto): puede conectarse y leer una vista, y nada más.

> **Error frecuente.** Conceder RESOURCE y esperar que el usuario pueda crear vistas. RESOURCE no incluye CREATE VIEW, así que `CREATE VIEW` falla con ORA-01031 (privilegios insuficientes): concede CREATE VIEW de forma explícita.

TCL: transacciones

TCL: transacciones

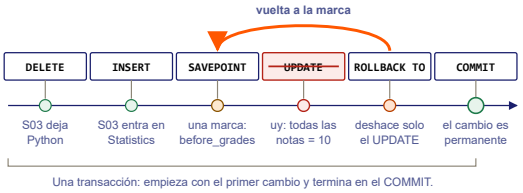
Una transacción agrupa sentencias que deben tener éxito o fallar juntas. COMMIT las confirma (las hace permanentes), ROLLBACK las deshace y un SAVEPOINT (punto de guardado) marca un punto al que se puede

volver.

- `SAVEPOINT nombre` … `ROLLBACK TO SAVEPOINT nombre` deshace solo el trabajo posterior al punto de guardado.
- Mantén las transacciones cortas: mientras una está abierta, puede retener bloqueos que hacen esperar a las demás.
- En Oracle los lectores nunca bloquean a los escritores (conserva versiones antiguas de las filas) y, por defecto, una consulta ve los datos confirmados en el momento en que empezó.

Ejemplo que puedes ejecutar en la web

```
DELETE FROM enrolment WHERE student_id = 'S03' AND course_id =
'C11';
INSERT INTO enrolment VALUES ('S03', 'C20', NULL);
SAVEPOINT before_grades;
UPDATE enrolment SET grade = 10;              -- ¡uy!: todas
las filas
ROLLBACK TO SAVEPOINT before_grades;
COMMIT;
SELECT * FROM enrolment WHERE student_id = 'S03';
DELETE FROM enrolment;
SELECT COUNT(*) AS rows_inside_transaction FROM enrolment;
ROLLBACK;
SELECT COUNT(*) AS rows_after_rollback FROM enrolment;
```



ROLLBACK TO SAVEPOINT deshace solo el trabajo posterior a la marca; COMMIT hace permanente el resto.

> **Ejemplo.** El segundo bloque cambia una nota por error y vuelve al punto de guardado: el cambio de asignatura se conserva, la nota errónea no.

> **Error frecuente.** Ejecutar un DDL en mitad de una transacción y esperar poder deshacer. Una sentencia DDL confirma todo lo anterior, así que los cambios previos ya no se pueden deshacer.

Commit implícito y autocommit

Una transacción puede terminar sin que tú escribas COMMIT: toda sentencia DDL o DCL confirma antes de ejecutarse, y también lo hace la salida normal del cliente. Una caída o una conexión perdida, en cambio, deshacen la transacción.

- Ejecuta el DDL al principio de un script o justo después de un COMMIT, nunca en mitad de cambios que quizá quieras deshacer.
- TRUNCATE también es DDL: confirma la transacción abierta antes de vaciar la tabla.
- Otros SGBD son distintos: PostgreSQL puede deshacer la mayoría del DDL dentro de una transacción; Oracle y MySQL no.

Cómo termina una transacción

Suceso	Qué pasa con la transacción abierta
`COMMIT`	Sus cambios pasan a ser permanentes y visibles para las demás sesiones
`ROLLBACK`	Sus cambios se deshacen
`ROLLBACK TO SAVEPOINT s`	Solo se deshacen los cambios posteriores a s; la transacción sigue abierta
Una sentencia DDL o DCL	COMMIT implícito, antes de la sentencia y después
Una sentencia que falla	Solo se deshace esa sentencia; la transacción sigue abierta
Una salida normal (EXIT en SQL*Plus)	COMMIT
Una caída o una conexión perdida	ROLLBACK

Ejemplo que puedes ejecutar en la web

```
-- 1. Algo de DML: la transacción está abierta
UPDATE course SET credits = 12 WHERE course_id = 'C30';
INSERT INTO department VALUES ('D03', 'Law');
-- 2. Una sentencia DDL en medio confirma todo lo anterior
CREATE TABLE note (txt VARCHAR2(40));
-- 3. Así que este ROLLBACK no tiene nada que deshacer
ROLLBACK;
SELECT course_id, credits FROM course WHERE course_id = 'C30';
SELECT * FROM department;
-- 4. Sin DDL en medio, ROLLBACK funciona
DELETE FROM enrolment WHERE course_id = 'C10';
```

```
ROLLBACK;
SELECT COUNT(*) AS c10_enrolments FROM enrolment WHERE course_id = 'C10';
```

> **Ejemplo.** El ejemplo de puntos de guardado del Tema 4: `UPDATE emp SET sal = 7000 WHERE ename = 'Garcia'`, `SAVEPOINT garcia_sal`, `UPDATE emp SET sal = 12000 WHERE ename = 'Perez'`, `ROLLBACK TO SAVEPOINT garcia_sal` y `COMMIT`: la subida de Pérez se deshace y la de García se conserva.

> **Error frecuente.** Probar un UPDATE y luego crear una tabla «para guardar los resultados» antes de decidir si te quedas con el UPDATE. El CREATE TABLE ya lo ha confirmado.

PL/SQL

Bloques PL/SQL, variables y SELECT INTO

PL/SQL es el lenguaje procedimental de Oracle: SQL más variables, condiciones, bucles y gestión de errores. El código se escribe en bloques: una sección DECLARE opcional, un cuerpo BEGIN … END y una sección EXCEPTION opcional.

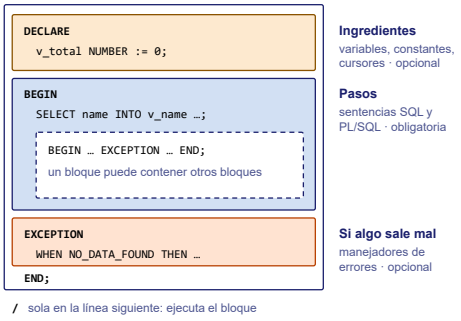
- `:=` asigna y `=` compara; cada sentencia termina en `;`.
- Pon prefijos a los nombres (`v_` variable, `c_` constante, `r_` registro) para que nunca choquen con nombres de columnas.
- Si SELECT INTO no encuentra ninguna fila lanza NO_DATA_FOUND; si encuentra varias, TOO_MANY_ROWS (mira la tarjeta de excepciones).

Oracle PL/SQL · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS enrolment CASCADE CONSTRAINTS PURGE;
DROP TABLE IF EXISTS student CASCADE CONSTRAINTS PURGE;
CREATE TABLE student (
  student_id VARCHAR2(3) PRIMARY KEY,
  name       VARCHAR2(40) NOT NULL
);
CREATE TABLE enrolment (
  student_id VARCHAR2(3) REFERENCES student (student_id),
  course_id  VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (student_id, course_id)
);
INSERT INTO student VALUES ('S01', 'Ana Ruiz');
INSERT INTO enrolment VALUES ('S01', 'C10', 8.5);
INSERT INTO enrolment VALUES ('S01', 'C20', 7.0);
COMMIT;

DECLARE
  c_pass    CONSTANT NUMBER := 5;
  r_student student%ROWTYPE;              -- una fila entera
de student
  v_average enrolment.grade%TYPE;         -- el tipo de la
columna grade
  v_courses PLS_INTEGER;
BEGIN
  SELECT * INTO r_student FROM student WHERE student_id =
'S01';
  SELECT AVG(grade), COUNT(*) INTO v_average, v_courses
  FROM enrolment
  WHERE student_id = r_student.student_id;

  DBMS_OUTPUT.PUT_LINE(r_student.name || ' takes ' ||
v_courses || ' courses.');
  DBMS_OUTPUT.PUT_LINE('Average: ' || TO_CHAR(v_average,
'FM90.00'));
  IF v_average >= c_pass THEN
    DBMS_OUTPUT.PUT_LINE('On track to pass.');
  END IF;
END;
/
```



Las partes de un bloque. Solo BEGIN … END es obligatorio.

> **Ejemplo.** El bloque del ejemplo lee un estudiante y su media en variables e imprime una frase sobre él, algo que un único SELECT no puede hacer.

> **Error frecuente.** Dar a una variable el nombre de una columna (`name`). En `WHERE name = name` los dos lados son la columna, así que la condición es cierta para todas las filas. Llámala `v_name`.

Control de flujo: IF, CASE y bucles

PL/SQL decide con IF … ELSIF … ELSE y CASE, y repite con tres bucles: el LOOP básico con EXIT WHEN, WHILE y el FOR numérico.

- Todo bucle necesita una salida: un LOOP básico sin EXIT no termina nunca.
- `CONTINUE WHEN condición;` salta a la siguiente vuelta.
- En los programas reales el bucle suele recorrer filas: para eso están los cursores (siguiente tarjeta).

Oracle PL/SQL · ejecútalo en FreeSQL

```
DECLARE
  v_grade   NUMBER := 7.5;
  v_label   VARCHAR2(20);
  v_balance NUMBER := 1000;
  v_years   PLS_INTEGER := 0;
BEGIN
  -- IF / ELSIF / ELSE: gana la primera rama cierta
  IF v_grade >= 9 THEN
    v_label := 'Outstanding';
  ELSIF v_grade >= 7 THEN
    v_label := 'Good';
  ELSIF v_grade >= 5 THEN
    v_label := 'Pass';
  ELSE
    v_label := 'Fail';
  END IF;
  DBMS_OUTPUT.PUT_LINE(v_grade || ' is ' || v_label);

  -- Sentencia CASE sobre un valor
  CASE v_label
    WHEN 'Fail' THEN DBMS_OUTPUT.PUT_LINE('See you at the resit.');
    WHEN 'Pass' THEN DBMS_OUTPUT.PUT_LINE('Just made it.');
    ELSE DBMS_OUTPUT.PUT_LINE('Well done!');
  END CASE;

  -- FOR: un bucle que cuenta
  FOR i IN 1 .. 5 LOOP
    DBMS_OUTPUT.PUT_LINE('7 x ' || i || ' = ' || 7 * i);
  END LOOP;

  -- WHILE: años hasta que 1.000 euros al 5 % anual lleguen a 1.500
  WHILE v_balance < 1500 LOOP
    v_balance := v_balance * 1.05;
    v_years := v_years + 1;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(v_years || ' years: ' || ROUND(v_balance, 2) || ' euros');

  -- LOOP ... EXIT WHEN: una cuenta atrás
  LOOP
    DBMS_OUTPUT.PUT_LINE('Countdown: ' || v_years);
    v_years := v_years - 3;
    EXIT WHEN v_years <= 0;
  END LOOP;
END;
/
```

> **Ejemplo.** Calificar una nota: `IF v_grade >= 9 THEN … ELSIF v_grade >= 7 THEN … ELSIF v_grade >= 5 THEN … ELSE … END IF;`.

> **Error frecuente.** Escribir `ELSE IF` o `ELSEIF`. PL/SQL quiere `ELSIF`; `ELSE IF` abre un IF nuevo que necesita su propio `END IF;`.

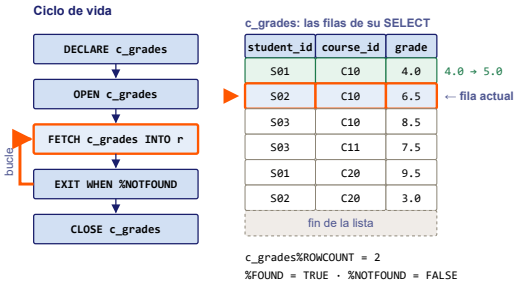
Cursores: recorrer las filas una a una

Un cursor señala las filas de una consulta para que PL/SQL las procese de una en una. Oracle abre un cursor implícito para cada sentencia SQL; declaras un cursor explícito cuando necesitas visitar una a una las filas de un SELECT.

- Tras cualquier INSERT, UPDATE o DELETE, el cursor implícito dice cuántas filas han cambiado (`SQL%ROWCOUNT`) y si ha cambiado alguna (`SQL%FOUND`).
- `FOR UPDATE` bloquea las filas hasta el COMMIT o el ROLLBACK, para que nadie las cambie a medias; confirma después de cerrar el cursor, no dentro del bucle.
- En un bucle FOR de cursor el registro `r` se declara solo y el cursor siempre se cierra, incluso tras un error.

Atributos de cursor

Atributo	Significa	Uso típico
`c%ISOPEN`	El cursor está abierto	`IF NOT c%ISOPEN THEN OPEN c; END IF;`
`c%FOUND`	El último FETCH devolvió una fila	Procesar la fila
`c%NOTFOUND`	El último FETCH no encontró nada: el final de la lista	`EXIT WHEN c%NOTFOUND;`
`c%ROWCOUNT`	Filas recuperadas hasta ahora	Numerar las líneas de un informe
`SQL%ROWCOUNT`	Filas cambiadas por la última sentencia SQL (cursor implícito)	`DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows')`



Un bucle FOR de cursor hace el OPEN, FETCH, EXIT y CLOSE por ti.

El cursor es un dedo sobre las filas de su SELECT: cada FETCH lo baja una fila.

Oracle PL/SQL · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS enrolment CASCADE CONSTRAINTS PURGE;
CREATE TABLE enrolment (
  student_id VARCHAR2(3),
  course_id  VARCHAR2(3),
  grade      NUMBER(4,2) CHECK (grade BETWEEN 0 AND 10),
  PRIMARY KEY (student_id, course_id)
);
INSERT INTO enrolment VALUES ('S01', 'C10', 4.0);
INSERT INTO enrolment VALUES ('S02', 'C10', 6.5);
INSERT INTO enrolment VALUES ('S03', 'C10', 8.5);
INSERT INTO enrolment VALUES ('S03', 'C11', 7.5);
INSERT INTO enrolment VALUES ('S01', 'C20', 9.5);
INSERT INTO enrolment VALUES ('S02', 'C20', 3.0);
COMMIT;

-- 1. Cursor explícito: DECLARE -> OPEN -> FETCH en un bucle -> CLOSE
DECLARE
  CURSOR c_grades IS
    SELECT student_id, course_id, grade
    FROM enrolment
    ORDER BY course_id, student_id
    FOR UPDATE OF grade;
  r     c_grades%ROWTYPE;
  v_new enrolment.grade%TYPE;
BEGIN
  OPEN c_grades;
  LOOP
    FETCH c_grades INTO r;
    EXIT WHEN c_grades%NOTFOUND;              -- el dedo ha llegado al final de la lista
    v_new := CASE
               WHEN r.course_id = 'C10' AND r.grade < 5 THEN r.grade + 1
               WHEN r.course_id = 'C10' AND r.grade < 7 THEN r.grade + 0.5
```

```
               WHEN r.course_id = 'C20'                THEN r.grade + 1
               ELSE r.grade
             END;
    v_new := LEAST(v_new, 10);                -- nunca por encima de 10
    IF v_new <> r.grade THEN
      UPDATE enrolment SET grade = v_new WHERE CURRENT OF c_grades;
      DBMS_OUTPUT.PUT_LINE(c_grades%ROWCOUNT || '. ' || r.student_id || ' ' || r.course_id || ': ' || r.grade || '
-> ' || v_new);
    END IF;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(c_grades%ROWCOUNT || ' rows read');
  CLOSE c_grades;
  COMMIT;
END;
/

-- 2. El bucle FOR de cursor: OPEN, FETCH, EXIT y CLOSE son automáticos
DECLARE
  CURSOR c_list IS
    SELECT student_id, course_id, grade FROM enrolment ORDER BY course_id, student_id;
BEGIN
  FOR r IN c_list LOOP
    DBMS_OUTPUT.PUT_LINE(r.course_id || ' ' || r.student_id || ': ' || r.grade);
  END LOOP;
  -- El cursor implícito de un UPDATE normal
  UPDATE enrolment SET grade = LEAST(grade + 0.25, 10) WHERE course_id = 'C11';
  DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' Python grade(s) raised');
  COMMIT;
END;
/

-- 3. El ajuste del bloque 1 como UNA sola sentencia de conjunto (prefiérela si basta una fórmula):
-- UPDATE enrolment
-- SET grade = LEAST(10, grade + CASE
--                                WHEN course_id = 'C10' AND grade < 5 THEN 1
--                                WHEN course_id = 'C10' AND grade < 7 THEN 0.5
--                                WHEN course_id = 'C20' THEN 1
--                                ELSE 0
--                              END);
```

Mira el vídeo en la web, en la sección SQL.

> **Ejemplo.** El 4 de S01 en Databases pasa a 5, el 6,5 de S02 pasa a 7 y el 8,5 se queda igual; en Statistics todo el mundo gana un punto, pero el 9,5 se queda en 10.

> **Error frecuente.** Escribir un bucle con cursor cuando bastaría **un único UPDATE de conjunto**. Si cada fila solo necesita una fórmula sobre sus propios valores, `UPDATE enrolment SET grade = LEAST(10, grade + CASE … END)` cambia toda la tabla en una sentencia, y mucho más deprisa. Reserva los cursores para el trabajo que SQL no puede expresar en una sentencia: varios pasos por fila, llamadas a procedimientos, mensajes o decisiones que dependen de filas anteriores.

Excepciones: gestionar los errores

Cuando una sentencia falla dentro de PL/SQL, Oracle lanza una excepción y salta a la sección EXCEPTION del bloque. Ahí capturas los errores por su nombre y los tratas, y el programa sigue en lugar de detenerse.

- Pon primero los manejadores específicos y WHEN OTHERS al final.
- Una excepción que un bloque no trata pasa al bloque que lo contiene y, al final, a quien lo llamó.
- Dentro de un bucle, un bloque anidado trata el error de una fila y el bucle continúa con la siguiente.

Excepciones predefinidas habituales

Excepción	Error	Se lanza cuando
`NO_DATA_FOUND`	ORA-01403	Un SELECT INTO no devuelve ninguna fila
`TOO_MANY_ROWS`	ORA-01422	Un SELECT INTO devuelve más de una fila
`ZERO_DIVIDE`	ORA-01476	Se divide un número entre cero

Excepción	Error	Se lanza cuando
DUP_VAL_ON_INDEX	ORA-00001	Un INSERT o UPDATE repite un valor de clave primaria o UNIQUE
VALUE_ERROR	ORA-06502	Un valor no cabe en su variable (demasiado largo, tipo incorrecto)
INVALID_NUMBER	ORA-01722	En SQL se usa como número un texto que no lo es

Oracle PL/SQL · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS student CASCADE CONSTRAINTS PURGE;
CREATE TABLE student (
  student_id VARCHAR2(3) PRIMARY KEY,
  name       VARCHAR2(40) NOT NULL,
  credits    NUMBER(3) DEFAULT 0 NOT NULL
);
INSERT INTO student VALUES ('S01', 'Ana Ruiz', 60);
INSERT INTO student VALUES ('S02', 'Luis Gil', 0);
COMMIT;

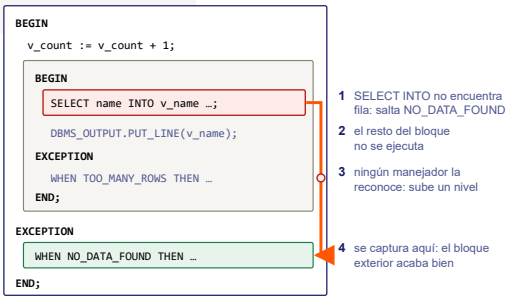
DECLARE
  v_name    student.name%TYPE;
  v_credits student.credits%TYPE;
  e_too_many_credits EXCEPTION;               -- una excepción
definida por el usuario
BEGIN
  BEGIN                                       -- NO_DATA_FOUND
    SELECT name INTO v_name FROM student WHERE student_id =
'S99';
  EXCEPTION
    WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('No student
S99');
  END;

  BEGIN                                       -- TOO_MANY_ROWS
    SELECT name INTO v_name FROM student;
  EXCEPTION
    WHEN TOO_MANY_ROWS THEN DBMS_OUTPUT.PUT_LINE('More than
one student: use a cursor');
  END;

  BEGIN                                       -- ZERO_DIVIDE:
Luis tiene 0 créditos
    SELECT credits INTO v_credits FROM student WHERE
student_id = 'S02';
    DBMS_OUTPUT.PUT_LINE('Hours per credit: ' || 150 /
v_credits);
  EXCEPTION
    WHEN ZERO_DIVIDE THEN DBMS_OUTPUT.PUT_LINE('Cannot divide
by zero credits');
  END;

  BEGIN                                       --
DUP_VAL_ON_INDEX: S01 ya existe
    INSERT INTO student (student_id, name) VALUES ('S01', 'Ana
again');
  EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN DBMS_OUTPUT.PUT_LINE('S01
already exists');
  END;

  BEGIN                                       -- una regla de
negocio con una excepción propia
    SELECT credits INTO v_credits FROM student WHERE
student_id = 'S01';
    IF v_credits + 30 > 78 THEN
      RAISE e_too_many_credits;
```



Si ningún bloque la maneja, el error llega a quien llamó y el script se detiene.

Una excepción salta al manejador más cercano que la reconoce; si ninguno lo hace, sube al bloque que lo contiene.

```
    END IF;
  EXCEPTION
    WHEN e_too_many_credits THEN DBMS_OUTPUT.PUT_LINE('No more t
  END;

  BEGIN                                    -- tu propio númer
    RAISE_APPLICATION_ERROR(-20001, 'Enrolment is closed');
  EXCEPTION
    WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Error ' || SQLCODE ||
  END;
END;
/
```

Ejemplo. Buscar un estudiante que no existe imprime «No student S99», y el script sigue con el paso siguiente.

Error frecuente. Escribir `WHEN OTHERS THEN NULL;`. Se traga todos los errores en silencio: el programa parece funcionar y los datos están mal. Como mínimo, registra SQLERRM o vuelve a lanzar el error con `RAISE;`.

Procedimientos y funciones almacenados

Un procedimiento es un bloque PL/SQL con nombre, guardado en la base de datos, que hace algo. Una función devuelve un valor y se puede usar dentro de SQL. Los dos reciben parámetros.

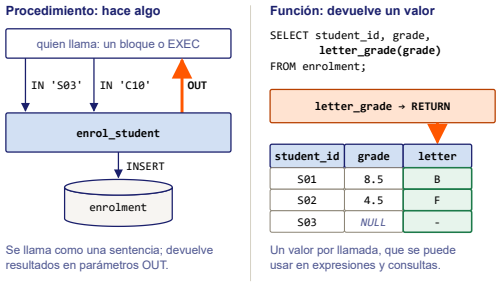
- Si compila con errores, Oracle la guarda como INVALID; `SHOW ERRORS` (o la vista `USER_ERRORS`) dice por qué.
- Una función que se llama desde SQL no debería modificar datos.
- Pon a los parámetros el prefijo `p_` para que nunca choquen con nombres de columnas.

Oracle PL/SQL · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS enrolment CASCADE CONSTRAINTS PURGE;
CREATE TABLE enrolment (
  student_id VARCHAR2(3),
  course_id  VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (student_id, course_id)
);
INSERT INTO enrolment VALUES ('S01', 'C10', 8.5);
INSERT INTO enrolment VALUES ('S02', 'C10', 4.5);
COMMIT;

-- Una función: una calculadora que devuelve un valor
CREATE OR REPLACE FUNCTION letter_grade (p_grade IN NUMBER)
RETURN VARCHAR2 IS
BEGIN
  RETURN CASE
           WHEN p_grade IS NULL THEN '-'
           WHEN p_grade >= 9 THEN 'A'
           WHEN p_grade >= 7 THEN 'B'
           WHEN p_grade >= 5 THEN 'C'
           ELSE 'F'
         END;
END letter_grade;
/

-- Un procedimiento: una máquina expendedora que hace algo y
devuelve un mensaje
CREATE OR REPLACE PROCEDURE enrol_student (
  p_student IN  VARCHAR2,
  p_course  IN  VARCHAR2,
  p_message OUT VARCHAR2
) IS
BEGIN
  INSERT INTO enrolment (student_id, course_id) VALUES
(p_student, p_course);
  p_message := p_student || ' enrolled in ' || p_course;
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
```



Un procedimiento se llama como una sentencia; una función devuelve un valor y se puede usar en SQL.

```
    p_message := p_student || ' was already enrolled in ' || p_c
END enrol_student;
/

-- Las dos llamadas desde un bloque...
DECLARE
  v_msg VARCHAR2(100);
BEGIN
  enrol_student('S03', 'C10', v_msg);
  DBMS_OUTPUT.PUT_LINE(v_msg);
  enrol_student('S01', 'C10', v_msg);
  DBMS_OUTPUT.PUT_LINE(v_msg);
  DBMS_OUTPUT.PUT_LINE('8.5 is a ' || letter_grade(8.5));
  COMMIT;
END;
/

-- ...y la función desde SQL
SELECT student_id, grade, letter_grade(grade) AS letter
FROM enrolment
ORDER BY student_id;
```

Mira el vídeo en la web, en la sección SQL.

> **Ejemplo.** `letter_grade(8.5)` devuelve `B`, y `SELECT student_id, grade, letter_grade(grade) FROM enrolment` la aplica a cada fila.

> **Error frecuente.** Escribir un procedimiento donde hace falta una función, o al revés. Si quieres un valor para usarlo en una expresión o en una consulta, escribe una función; si quieres una acción, un procedimiento.

Disparadores (triggers)

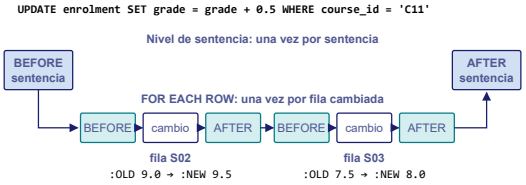
Un *disparador* es código PL/SQL que Oracle ejecuta *automáticamente* cuando le pasa algo a una tabla: antes o después de un INSERT, UPDATE o DELETE, una vez por sentencia o una vez *por cada fila*.

- En un INSERT los valores `:OLD` son NULL; en un DELETE lo son los `:NEW`.
- Los usos típicos son las auditorías, los valores derivados y las reglas entre tablas; las reglas sencillas van en restricciones.
- Un disparador se ejecuta dentro de la transacción de su sentencia: si la sentencia se deshace, el trabajo del disparador también.

Oracle PL/SQL · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS grade_audit PURGE;
DROP TABLE IF EXISTS enrolment CASCADE CONSTRAINTS PURGE;
CREATE TABLE enrolment (
  student_id VARCHAR2(3),
  course_id  VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (student_id, course_id)
);
CREATE TABLE grade_audit (
  audit_id   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  student_id VARCHAR2(3),
  course_id  VARCHAR2(3),
  old_grade  NUMBER(4,2),
  new_grade  NUMBER(4,2),
  changed_by VARCHAR2(128) DEFAULT USER,
  changed_at DATE DEFAULT SYSDATE
);
INSERT INTO enrolment VALUES ('S01', 'C10', 4.5);
INSERT INTO enrolment VALUES ('S02', 'C10', 6.0);
COMMIT;

-- BEFORE ... FOR EACH ROW: comprueba y ajusta el valor nuevo
CREATE OR REPLACE TRIGGER trg_grade_check
BEFORE INSERT OR UPDATE OF grade ON enrolment
```



Orden de disparo para un UPDATE que cambia dos filas.

```
FOR EACH ROW
BEGIN
  IF :NEW.grade > 10 THEN
    RAISE_APPLICATION_ERROR(-20010, 'A grade cannot be above 10'
  END IF;
  :NEW.grade := ROUND(:NEW.grade, 1);          -- guarda un solo
END;
/

-- AFTER ... FOR EACH ROW con WHEN: audita solo los cambios real
CREATE OR REPLACE TRIGGER trg_grade_audit
AFTER UPDATE OF grade ON enrolment
FOR EACH ROW
WHEN (NEW.grade <> OLD.grade OR (OLD.grade IS NULL AND NEW.grade
BEGIN
  INSERT INTO grade_audit (student_id, course_id, old_grade, new
  VALUES (:OLD.student_id, :OLD.course_id, :OLD.grade, :NEW.grad
END;
/

-- AFTER, de sentencia: una vez, cambien las filas que cambien
CREATE OR REPLACE TRIGGER trg_grade_notice
AFTER UPDATE OF grade ON enrolment
BEGIN
  DBMS_OUTPUT.PUT_LINE('Grades updated at ' || TO_CHAR(SYSDATE,
END;
/

UPDATE enrolment SET grade = grade + 0.54 WHERE course_id = 'C10
COMMIT;
SELECT * FROM enrolment;
SELECT student_id, old_grade, new_grade, changed_by, TO_CHAR(cha
FROM grade_audit;
```

Mira el vídeo en la web, en la sección SQL.

> **Ejemplo.** Cada cambio de una nota escribe una línea en `grade_audit` con quién, cuándo, la nota anterior y la nueva, sin que la aplicación haga nada.

> **Error frecuente.** Esconder lógica de negocio en disparadores, o consultar la misma tabla desde un disparador de fila. Las cadenas de acciones invisibles son difíciles de depurar, y leer la tabla que se está modificando falla con ORA-04091 (table is mutating). Mantén los disparadores pequeños: auditoría y comprobaciones que las restricciones no pueden expresar.

Paquetes

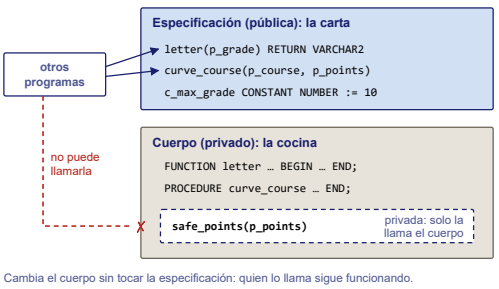
Un paquete agrupa procedimientos, funciones, constantes y cursores relacionados bajo un nombre. Tiene dos partes: la especificación, su interfaz pública, y el cuerpo, la implementación oculta.

- Cambiar solo el cuerpo no invalida los programas que llaman al paquete.
- Dos subprogramas de un paquete pueden llamarse igual si sus parámetros son distintos (sobrecarga).
- Concede EXECUTE sobre el paquete, no sobre cada procedimiento.

Oracle PL/SQL · ejecútalo en FreeSQL

```
DROP TABLE IF EXISTS enrolment CASCADE CONSTRAINTS PURGE;
CREATE TABLE enrolment (
  student_id VARCHAR2(3),
  course_id  VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (student_id, course_id)
);
INSERT INTO enrolment VALUES ('S01', 'C10', 4.5);
INSERT INTO enrolment VALUES ('S02', 'C10', 7.25);
COMMIT;

-- Especificación: la carta (pública)
```



Cambia el cuerpo sin tocar la especificación: quien lo llama sigue funcionando.

Quien llama solo ve la especificación; el cuerpo y sus ayudantes privados quedan ocultos.

```
CREATE OR REPLACE PACKAGE grades_pkg IS
  c_max_grade CONSTANT NUMBER := 10;
  FUNCTION  letter (p_grade IN NUMBER) RETURN VARCHAR2;
  PROCEDURE curve_course (p_course IN VARCHAR2, p_points IN NUMB
END grades_pkg;
/

-- Cuerpo: la cocina (detalles privados)
CREATE OR REPLACE PACKAGE BODY grades_pkg IS
  -- Privada: no está en la especificación, así que solo este cu
  FUNCTION safe_points (p_points IN NUMBER) RETURN NUMBER IS
  BEGIN
    RETURN LEAST(GREATEST(p_points, 0), 2);      -- un ajuste va
  END safe_points;

  FUNCTION letter (p_grade IN NUMBER) RETURN VARCHAR2 IS
  BEGIN
    RETURN CASE WHEN p_grade >= 9 THEN 'A' WHEN p_grade >= 7 THE
                WHEN p_grade >= 5 THEN 'C' ELSE 'F' END;
  END letter;

  PROCEDURE curve_course (p_course IN VARCHAR2, p_points IN NUMB
    v_points NUMBER := safe_points(p_points);
  BEGIN
    UPDATE enrolment
    SET grade = LEAST(grade + v_points, c_max_grade)
    WHERE course_id = p_course;
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' grades curved by ' ||
  END curve_course;
END grades_pkg;
/

BEGIN
  grades_pkg.curve_course('C10', 5);           -- pide 5; la reg
  COMMIT;
END;
/
SELECT student_id, grade, grades_pkg.letter(grade) AS letter FRO
```

> **Ejemplo.** `grades_pkg` publica `letter` y `curve_course`; la regla de que un ajuste está entre 0 y 2 puntos es una función privada que nadie fuera del paquete puede llamar ni saltarse.

> **Error frecuente.** Poner todas las utilidades en la especificación. Todo lo público se convierte en una promesa: otro código empieza a usarlo y ya no puedes cambiarlo con libertad. Publica solo lo que necesitan quienes llaman.

Índices

Qué es un índice

Un índice es una estructura aparte y ordenada (normalmente un árbol B+) que asocia los valores de algunas columnas con sus filas. Gracias a él, el SGBD puede buscar en lugar de recorrer toda la tabla.

- La clave primaria y cada restricción UNIQUE ya tienen un índice.
- En Oracle las claves ajenas **no** reciben uno automáticamente; indéxalas si combinas o filtras por ellas.

Ejemplo que puedes ejecutar en la web

```
EXPLAIN PLAN FOR SELECT * FROM customer WHERE email = 'user43210@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT * FROM customer WHERE email = 'user43210@mail.com';
CREATE INDEX idx_customer_email ON customer (email);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE email = 'user43210@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT * FROM customer WHERE email = 'user43210@mail.com';
```

Mira el vídeo en la web, en la sección SQL.

> **Error frecuente.** Pensar que un índice cambia el resultado. Solo cambia la rapidez con que el SGBD encuentra las filas; la consulta devuelve exactamente los mismos datos.

Índices compuestos y de cobertura

Un índice compuesto ordena por varias columnas, en orden. Ayuda a las consultas que filtran por sus columnas más a la izquierda. Un índice de cobertura contiene todas las columnas que necesita una consulta, de modo que no hace falta leer la tabla.

- Regla del prefijo izquierdo: `(a, b, c)` sirve para filtros sobre `a`, `a, b` y `a, b, c`.
- Un índice compuesto puede sustituir a varios de una sola columna.

Ejemplo que puedes ejecutar en la web

```
CREATE INDEX idx_city_date ON customer (city, signup_date);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE city = 'Madrid' AND signup_date >= DATE '2023-01-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE signup_date >= DATE '2023-01-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT city, signup_date FROM customer WHERE city = 'Sevilla';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT COUNT(*) AS madrid_since_2023 FROM customer WHERE city = 'Madrid' AND signup_date >= DATE '2023-01-01';
```

> **Error frecuente.** Crear `(signup_date, city)` para consultas que siempre fijan la ciudad y dan un rango de fechas. Con el rango primero, la segunda columna no puede acotar la búsqueda.

Lo que cuestan los índices

Los índices aceleran las lecturas pero ralentizan las escrituras: cada INSERT, UPDATE o DELETE debe actualizar también cada índice de la tabla. Además, ocupan disco y memoria.

Cuándo indexar una columna

Normalmente sí	Normalmente no
Claves ajenas usadas en combinaciones (JOIN)	Columnas que nunca se usan en WHERE, JOIN u ORDER BY
Columnas filtradas con = o rangos que devuelven pocas filas	Columnas de baja selectividad (sexo, sí/no)
Columnas usadas para ordenar resultados grandes (ORDER BY … LIMIT)	Tablas pequeñas que caben en pocos bloques
Claves de búsqueda de consultas frecuentes	Tablas con muchas escrituras y pocas lecturas (registros, logs)

Ejemplo que puedes ejecutar en la web

```
INSERT INTO plain_log
SELECT LEVEL, MOD(LEVEL, 997), 'action' || MOD(LEVEL, 13), DATE '2024-01-01' + MOD(LEVEL, 365)
FROM dual CONNECT BY LEVEL <= 20000;
INSERT INTO indexed_log
SELECT LEVEL, MOD(LEVEL, 997), 'action' || MOD(LEVEL, 13), DATE '2024-01-01' + MOD(LEVEL, 365)
FROM dual CONNECT BY LEVEL <= 20000;
SELECT index_name AS indexes_on_indexed_log FROM user_indexes WHERE table_name = 'INDEXED_LOG';
```

> **Error frecuente.** Indexar todas las columnas «por si acaso». Cada índice ralentiza todas las escrituras y el optimizador solo usa los útiles.

Tipos de índice

El árbol B es el tipo por defecto y sirve para igualdad, rangos y ordenación. Otros tipos resuelven problemas concretos: hash solo para igualdad, bitmap para pocos valores distintos, GIN para valores dentro de un documento o un array y BRIN para tablas enormes ordenadas de forma natural.

Principales tipos de índice

Tipo	Adecuado para	No sirve para

Tipo	Adecuado para	No sirve para
Árbol B / árbol B+	=, <, >, BETWEEN, ORDER BY, LIKE con prefijo 'abc%'	Buscar dentro de textos o arrays
Hash	Solo igualdad (=) (PostgreSQL, MySQL MEMORY)	Rangos y ordenación
Bitmap	Columnas con pocos valores distintos en tablas casi solo de lectura (almacenes de datos)	Tablas con muchas escrituras concurrentes
Clave inversa (Oracle)	Claves crecientes (números de secuencia) insertadas por muchas sesiones a la vez: reparte la hoja caliente de la derecha	Recorridos por rango: destruye el orden de la clave
GIN / invertido	Búsqueda de texto completo, claves JSON, elementos de arrays	Columnas escalares simples
BRIN	Tablas muy grandes ordenadas por inserción (fechas de un registro) (PostgreSQL)	Datos en orden aleatorio

Oracle · no se ejecuta aquí

```
CREATE INDEX orders_customer_idx ON orders (customer_id);               -- árbol B (por defecto)
CREATE BITMAP INDEX customer_city_bix ON customer (city);               -- pocos valores distintos, almacén de
datos
CREATE INDEX orders_id_rev ON orders (order_id) REVERSE;                -- clave inversa
CREATE INDEX customer_email_ci ON customer (LOWER(email));              -- basado en funciones
CREATE UNIQUE INDEX open_order_uq ON orders (CASE WHEN status = 'OPEN' THEN customer_id END);  -- único «parcial»
```

> **Error frecuente.** Crear un índice bitmap en una tabla que muchas sesiones actualizan a la vez. Cada cambio bloquea un rango entero de filas del mapa de bits, así que los escritores se esperan unos a otros.

Agrupamiento

Índices agrupados y no agrupados

En un índice agrupado (clustered), las propias filas de la tabla se guardan en el orden de la clave, dentro del índice. Una tabla solo puede tener uno. Todos los demás índices son no agrupados: guardan la clave más un puntero a la fila.

- Elige una clave de agrupamiento corta, que no cambie y que coincida con las consultas por rango frecuentes.
- Las claves aleatorias (como los UUID aleatorios) dispersan las inserciones por todo el índice agrupado; las claves crecientes se añaden al final.

Ejemplo que puedes ejecutar en la web

```
CREATE TABLE enrolment_by_course (
  course_id  VARCHAR2(3),
  student_id VARCHAR2(3),
  grade      NUMBER(4,2),
  PRIMARY KEY (course_id, student_id)
) ORGANIZATION INDEX;
INSERT INTO enrolment_by_course VALUES ('C20', 'S01', 7.0);
INSERT INTO enrolment_by_course VALUES ('C10', 'S03', 5.0);
INSERT INTO enrolment_by_course VALUES ('C10', 'S01', 8.5);
INSERT INTO enrolment_by_course VALUES ('C11', 'S02', 9.0);
INSERT INTO enrolment_by_course VALUES ('C10', 'S02', 6.5);
SELECT * FROM enrolment_by_course;           -- se guarda en el orden de la clave (sin ORDER BY el orden nunca
está garantizado)
EXPLAIN PLAN FOR SELECT * FROM enrolment_by_course WHERE course_id = 'C10';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

> **Error frecuente.** Confundir un **índice agrupado** (clustered index) con un **clúster de bases de datos**. Lo segundo es un grupo de servidores que trabajan juntos (replicación, alta disponibilidad), un significado de la palabra que no tiene nada que ver.

Clústeres de tablas en Oracle (y CLUSTER en PostgreSQL)

Un clúster de tablas de Oracle guarda en los mismos bloques las filas de varias tablas que comparten una clave (un departamento y sus empleados), lo que acelera las combinaciones por esa clave. PostgreSQL no tiene

algo así: su orden `CLUSTER` reescribe una vez la tabla en el orden de un índice.

Oracle · no se ejecuta aquí

```
CREATE CLUSTER dept_cluster (dept_id VARCHAR2(3));
CREATE INDEX dept_cluster_idx ON CLUSTER dept_cluster;      -- un clúster necesita su índice

CREATE TABLE department (
  dept_id VARCHAR2(3) PRIMARY KEY,
  name    VARCHAR2(30) NOT NULL
) CLUSTER dept_cluster (dept_id);

CREATE TABLE course (
  course_id VARCHAR2(3) PRIMARY KEY,
  title     VARCHAR2(40) NOT NULL,
  dept_id   VARCHAR2(3) NOT NULL REFERENCES department (dept_id)
) CLUSTER dept_cluster (dept_id);                           -- en los mismos bloques que su departamento
```

> **Error frecuente.** Agrupar tablas que se actualizan constantemente o que se leen a menudo por separado. Los clústeres solo compensan con datos estables que se leen juntos.

Particionado

Particionado horizontal

El particionado divide una tabla grande en trozos más pequeños, las particiones, según una clave de particionado. Las consultas siguen usando el nombre de la tabla única; el SGBD envía cada fila a su partición.

Cómo elegir el método

Método	Clave típica	Úsalo cuando
Por rango	Fecha, rangos de identificadores	Las consultas y la retención de datos van por tiempo
Por lista	País, región, estado	Unos pocos valores conocidos separan los datos
Por hash	Identificador de cliente	Necesitas tamaños parecidos y no hay un rango natural

Oracle · no se ejecuta aquí

```
CREATE TABLE orders (
  order_id    NUMBER(10),
  customer_id NUMBER(10) NOT NULL,
  order_date  DATE NOT NULL,
  total       NUMBER(10,2) NOT NULL
)
PARTITION BY RANGE (order_date) (
  PARTITION p2025_01 VALUES LESS THAN (DATE '2025-02-01'),
  PARTITION p2025_02 VALUES LESS THAN (DATE '2025-03-01'),
  PARTITION p_max    VALUES LESS THAN (MAXVALUE)             -- filas fuera de todos los rangos
);

-- Oracle crea solo las particiones mensuales:
-- PARTITION BY RANGE (order_date) INTERVAL (NUMTOYMINTERVAL(1, 'MONTH')) (PARTITION p_first VALUES LESS THAN
(DATE '2025-01-01'))

CREATE INDEX orders_customer_ix ON orders (customer_id) LOCAL;   -- un segmento de índice por partición
```

> **Error frecuente.** Particionar una tabla pequeña. Por debajo de muchos millones de filas, un buen índice suele ser más sencillo e igual de rápido.

Poda de particiones y mantenimiento

Cuando el WHERE fija la clave de particionado, el optimizador lee solo las particiones que coinciden: la poda de particiones (partition pruning). Los datos antiguos se pueden eliminar entonces borrando una partición entera en lugar de millones de filas.

- Elige la clave de particionado a partir de los filtros más frecuentes.

- Un índice **local** tiene un segmento por partición y se mantiene válido al borrar una partición; un índice único debe incluir la clave de particionado para ser local.

Oracle · no se ejecuta aquí

```
EXPLAIN PLAN FOR SELECT SUM(total) FROM orders
WHERE order_date >= DATE '2025-02-01' AND order_date < DATE '2025-03-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- | Id | Operation              | Name   | Pstart | Pstop |
-- |  2 |  PARTITION RANGE SINGLE|        |      2 |     2 |    <- solo se lee una partición
-- |  3 |   TABLE ACCESS FULL    | ORDERS |      2 |     2 |

ALTER TABLE orders DROP PARTITION p2025_01 UPDATE INDEXES;   -- se elimina un mes entero de una vez
ALTER TABLE orders EXCHANGE PARTITION p2025_01 WITH TABLE orders_2025_01_archive;  -- …o se saca como tabla
```

> **Error frecuente.** Particionar por una columna por la que las consultas casi nunca filtran. Sin poda, cada consulta visita todas las particiones.

Particionado vertical y sharding

El particionado vertical divide las columnas de una tabla en dos tablas con la misma clave. El sharding (fragmentación horizontal entre servidores) reparte las filas entre distintos servidores, cada uno con su propia base de datos.

Tres formas de dividir

Técnica	Divide	Dónde están los trozos	Coste principal
Particionado horizontal	Filas	La misma base de datos	Las consultas sin la clave visitan todas las particiones
Particionado vertical	Columnas	La misma base de datos	Una combinación para leer la fila completa
Sharding	Filas	Servidores distintos	Combinaciones y transacciones entre shards

> **Error frecuente.** Recurrir al sharding antes de necesitarlo. Los índices, el particionado, las réplicas de lectura y un servidor más potente resuelven la mayoría de los problemas con mucha menos complejidad.

Eficiencia

Leer un plan de ejecución

SQL dice qué quieres; el optimizador decide cómo: qué índice, qué orden de combinación, qué algoritmo. EXPLAIN PLAN muestra ese plan, y en el plan es donde se entienden las consultas lentas.

- TABLE ACCESS FULL = leer todas las filas; INDEX RANGE SCAN / INDEX UNIQUE SCAN = bajar por un índice.
- SORT ORDER BY = una ordenación adicional; un índice en ese orden la evita.

Ejemplo que puedes ejecutar en la web

```
EXPLAIN PLAN FOR
SELECT c.city, SUM(o.total) FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT c.city, ROUND(SUM(o.total), 2) AS spent FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
CREATE INDEX idx_orders_customer ON orders (customer_id);
EXEC DBMS_STATS.GATHER_TABLE_STATS(USER, 'ORDERS');
EXPLAIN PLAN FOR
SELECT c.city, SUM(o.total) FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
SELECT c.city, ROUND(SUM(o.total), 2) AS spent FROM customer c JOIN orders o ON o.customer_id = c.customer_id
WHERE c.customer_id BETWEEN 100 AND 200 GROUP BY c.city;
```

> **Error frecuente.** Optimizar por intuición. Lee primero el plan: la parte lenta a menudo no está donde esperas.

Condiciones que pueden usar un índice

Un índice sobre una columna solo ayuda cuando la condición compara la columna tal cual. Envolverla en una función o un cálculo la oculta al índice: la consulta recorre la tabla.

Ejemplo que puedes ejecutar en la web

```
EXPLAIN PLAN FOR SELECT COUNT(*) FROM customer WHERE TO_CHAR(signup_date, 'YYYY') = '2023';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT COUNT(*) FROM customer WHERE signup_date >= DATE '2023-01-01' AND signup_date < DATE
'2024-01-01';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
EXPLAIN PLAN FOR SELECT * FROM customer WHERE LOWER(email) = 'user77@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
CREATE INDEX idx_email_lower ON customer (LOWER(email));
EXPLAIN PLAN FOR SELECT * FROM customer WHERE LOWER(email) = 'user77@mail.com';
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

> **Error frecuente.** Añadir un índice y dar por hecho que la consulta lo usará. Comprueba el plan; basta una función sobre la columna para que el índice no sirva.

Escribir consultas eficientes

Pide a la base de datos exactamente lo que necesitas, en una sola sentencia orientada a conjuntos, y deja que haga el trabajo cerca de los datos: filtra, combina y agrega en SQL, no fila a fila en la aplicación.

- Selecciona solo las columnas que usas; `SELECT *` lee y envía más datos e impide que los índices de cobertura ayuden.
- Usa `EXISTS` para preguntar «¿hay alguno?» en lugar de `COUNT(*) > 0`: puede detenerse en la primera coincidencia.
- Pagina con `ORDER BY … FETCH FIRST n ROWS ONLY` (`OFFSET m ROWS` para saltar filas), idealmente sobre una columna indexada.
- Combina y filtra por claves indexadas, y mantén las transacciones cortas.

Ejemplo que puedes ejecutar en la web

```
-- «¿Ha hecho algún pedido el cliente 42?»: EXISTS puede detenerse en el primer pedido
SELECT CASE WHEN EXISTS (SELECT 1 FROM orders WHERE customer_id = 42) THEN 'yes' ELSE 'no' END AS has_orders FROM
dual;
-- Una sentencia orientada a conjuntos en lugar de un bucle sobre cada pedido
UPDATE orders SET total = ROUND(total * 0.9, 2) WHERE order_date < DATE '2023-02-01';
COMMIT;
-- Solo las columnas y filas necesarias, agregadas en la base de datos
SELECT c.city, COUNT(*) AS orders, ROUND(SUM(o.total), 2) AS spent
FROM orders o JOIN customer c ON c.customer_id = o.customer_id
WHERE o.order_date BETWEEN DATE '2024-01-01' AND DATE '2024-12-31'
GROUP BY c.city
ORDER BY spent DESC
FETCH FIRST 5 ROWS ONLY;
```

> **Ejemplo.** La última consulta del ejemplo responde a «las 5 ciudades con más gasto en 2024» con una sola sentencia, en lugar de cargar todos los pedidos en el programa.

> **Error frecuente.** Traer una tabla entera para filtrarla en la aplicación. La base de datos puede usar índices y enviar solo las filas que coinciden; el programa no puede hacer ninguna de las dos cosas.